



Django

Capítulo 7 Trabalhando com Templates

Django

Capítulo 7 - Trabalhando com Templates

Um Template contém as partes estáticas da saída HTML desejada, bem como alguma sintaxe especial descrevendo como o conteúdo dinâmico será inserido. É uma estrutura html que pode ter arquivos estáticos, estilos(css) e scripts(js), e uma sintaxe que, com python, permite se moldar a uma lista ou a um objeto do banco de dados. Nessa apresentação faremos um layout padrão para aplicação, a página de listagem de produtos na home e na página inicial de produtos e a pagina de edição de um produto.

Django

Capítulo 7 - Trabalhando com Templates

Inicialmente será criado o arquivo base. Ele será o “pai” de todos os templates e é nele que será adicionado os arquivos css e js globais, o cabeçalho e o rodapé do site, ou seja o estilo e estrutura comum entre as páginas do site, layout. Crie uma pasta chamada templates dentro do diretório loja e crie dentro dela um arquivo chamado base.html. Primeira parte do conteúdo do loja/templates/**base.html**:

```
{% load static %}
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">
```

Django

Capítulo 7 - Trabalhando com Templates

Segunda parte do conteúdo do loja/templates/base.html:

```
{% block styles %}{% endblock %}
<title>Loja - {% block title %}{% endblock %}</title>
</head>
<body>
<header></header>
{% block content %}{% endblock %}
<footer></footer>
{% block scripts %}{% endblock %}
</body>
</html>
```

loja > templates > <> base.html > ...

```
1  {% load static %}
2  <!DOCTYPE html>
3  <html lang="pt-BR">
4  <head>
5      <meta charset="utf-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
7      {% block styles %}{% endblock %}
8      <title>Loja - {% block title %}{% endblock %}</title>
9  </head>
10 <body>
11     <header></header>
12     {% block content %}{% endblock %}
13     <footer></footer>
14     {% block scripts %}{% endblock %}
15 </body>
16 </html>
```

Django

Capítulo 7 - Trabalhando com Templates

Explicando as tags adicionadas ao html que fazem parte do template:

`{% load static %}`: nessa linha estamos dizendo que o template será capaz de ler qualquer arquivo que esteja dentro da pasta static . Essa pasta ainda não foi criada.

`{% block %}{% endblock %}`: essa tag permite que criemos blocos dentro de um template estruturando a herança de template. Na herança de template, o template filho estende o template pai dentro dele e assim todo conteúdo e todos os blocos que existem dentro do pai são mesclados no filho.

`{% block content %}{% endblock %}`: onde será inserido o conteúdo de html

`{% block styles%}{% endblock %}`: onde será inserido conteúdos de estilo(css)

`{% block content %}{% endblock %}`: onde será inserido conteúdos de script(js)

Django

Capítulo 7 - Trabalhando com Templates

Adicione as linhas em destaque dentro da tag head do arquivo loja/templates/base.html para incluir o **Bootstrap** e o **Font Awesome** ao template base

```
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
    <!-- Adicione as linhas a seguir -->
    <link rel="stylesheet"
        href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/
        font-awesome/4.7.0/css/font-awesome.min.css">
    <link rel="stylesheet" href="{% static 'css/global.css' %}">
    {% block styles %}{% endblock %}
    <title>Loja - {% block title %}{% endblock %}</title>
</head>
```

Django

Capítulo 7 - Trabalhando com Templates

Adicione o código em destaque a seguir dentro da tag header do arquivo loja/templates/base.html (Parte 1)

```
<header>
```

```
    <nav class="navbar justify-content-end navbar-expand-lg">
      <ul class="nav">
        <li class="nav-item"><a class="nav-link" href="/"><i class="fa fa-home"></i>
Home</a></li>
        <li class="nav-item"><a class="nav-link" href="/categoria"><i class="fa
fa-book"></i>Categoria</a></li>
        <li class="nav-item"><a class="nav-link" href="/fabricante"><i class="fa
fa-book"></i> Fabricante</a></li>
        <li class="nav-item"><a class="nav-link" href="/produto"><i class="fa
fa-book"></i> Produto</a></li>
```

Django

Capítulo 7 - Trabalhando com Templates

Adicione o código em destaque a seguir dentro da tag header do arquivo loja/templates/base.html (Parte 2)

```
<li class="nav-item"><a class="nav-link" href="#"><i class="fa fa-sign-in"></i>
Entrar</a></li>
  <li class="nav-item"><a class="nav-link" href="#"><i class="fa fa-edit"></i>
Registrar</a></li>
</ul>
</nav>
</header>
```


Django

Capítulo 7 - Trabalhando com Templates

Adicione o código em destaque a seguir dentro da tag footer do arquivo loja/templates/**base.html**

```
<footer class="page-footer font-small blue">  
    <div class="footer-copyright text-center py-3">©2020 Copyright:  
        <a href="https://www.casadocodigo.com.br/"> Casadocodigo.com.br</a>  
    </div>  
</footer>
```

Django

Capítulo 7 - Trabalhando com Templates

Adicione os scripts em destaque a seguir após a tag footer e antes do do bloco {% block scripts %}{% endblock %} do arquivo loja/templates/**base.html**

```
</footer>
<script src="https://code.jquery.com/jquery-3.4.1.slim.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/popper.js@1.16.0/dist/umd/popper.min.js"></script>
<script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/js/bootstrap.min.js"></scrip
t>
{% block scripts %}{% endblock %}
</body>
</html>
```

Django

Capítulo 7 - Trabalhando com Templates

```
loja > templates > > base.html > ...
1  {% load static %}
2  <!DOCTYPE html>
3  <html Lang="pt-BR">
4  <head>
5      <meta charset="utf-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
7      {% block styles %}{% endblock %}
8      <title>Loja - {% block title %}{% endblock %}</title>
9      <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/css/bootstrap.min.css">
10     <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/font-awesome/4.7.0/css/font-awesome.min.css">
11     <link rel="stylesheet" href="{% static 'css/global.css' %}">
12 </head>
13 <body>
14     <header>
15         <nav class="navbar justify-content-end navbar-expand-lg">
16             <ul class="nav">
17                 <li class="nav-item"><a class="nav-link" href="/"><i class="fa fa-home"></i> Home</a></li>
18                 <li class="nav-item"><a class="nav-link" href="/categoria"><i class="fa fa-book"></i> Categoria</a></li>
19                 <li class="nav-item"><a class="nav-link" href="/fabricante"><i class="fa fa-book"></i> Fabricante</a></li>
20                 <li class="nav-item"><a class="nav-link" href="/produto"><i class="fa fa-book"></i> Produto</a></li>
21                 <li class="nav-item"><a class="nav-link" href="#"><i class="fa fa-sign-in"></i> Entrar</a></li>
22                 <li class="nav-item"><a class="nav-link" href="#"><i class="fa fa-edit"></i> Registrar</a></li>
23             </ul>
24         </nav>
25     </header>
26     {% block content %}{% endblock %}
27     <footer>
28         <div class="footer-copyright text-center py-3">©2020 Copyright:
29         <a href="https://www.casadocodigo.com.br/"> Casadocodigo.com.br</a>
30         </div>
31     </footer>
32     <script src="https://code.jquery.com/jquery-3.4.1.slim.min.js"></script>
33     <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.0/dist/umd/popper.min.js"></script>
34     <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.4.1/js/bootstrap.min.js"></script>
35     {% block scripts %}{% endblock %}
36 </body>
37 </html>
```

O primeiro bloco, carrega o bootstrap, o font awesome e o css global.css da pasta css, dentro da pasta static.

O segundo bloco contém os links para as futuras páginas do nosso projeto.

O terceiro contém apenas um link para o site do nosso livro de referência

O quarto importa funcionalidades do bootstrap, jquery e popper

Django

Capítulo 7 - Trabalhando com Templates

Agora vamos criar a primeira página de fato, crie o arquivo `home.html` que é a página de entrada do site. Crie uma pasta chamada `home` dentro do diretório `templates` e, dentro dela, crie o arquivo `home.html`. Parte 1 do conteúdo do `loja/templates/home/home.html`:

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
    <link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
    <div id="content">Olá mundo</div>
{% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do conteúdo do loja/templates/home/**home.html**:

```
{% block scripts %}
    <script src="{% static 'js/home.js' %}"></script>
{% endblock %}
```

loja > templates > home > <> home.html > ...

```
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Início{% endblock %}
4  {% block styles %}
5      <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8      <div id="content">Olá mundo</div>
9  {% endblock %}
10 {% block scripts %}
11     <script src="{% static 'js/home.js' %}"></script>
12 {% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Explicando a sintaxe que faz parte do template:

{% extends "base.html" %}: indica que o {% block content %}{% endblock %} desse html vai ser posicionado dentro {% block content %}{% endblock %} do arquivo base.html, ou seja o home.html herdará os css,js e estrutura html do base.html

{% block title %}Início{% endblock %}: Adiciona um título para a página no navegador

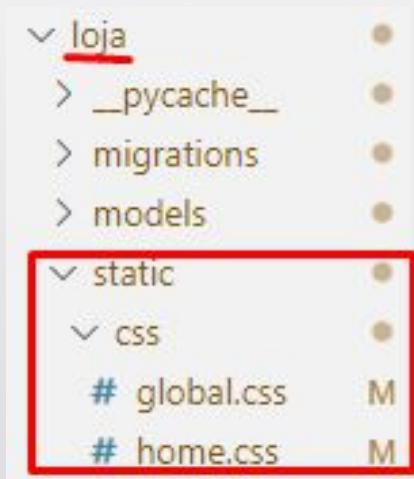
{% block styles %} <link rel="stylesheet" href="{% static 'css/home.css' %}">{% endblock %}: Carrega os estilos exclusivos desta página

{% block scripts %}<script src="{% static 'js/home.js' %}"></script>{% endblock %}: Carrega os scripts exclusivos desta página. O arquivo Javascript home.js não será inserido nesse momento.

Django

Capítulo 7 - Trabalhando com Templates

Agora vamos criar as pastas e arquivos dos nossos estilos requisitados. Primeiro crie uma pasta static na raiz da pasta loja. Após criar a pasta static, crie dentro dela uma pasta chamada css. e dentro da pasta css crie dois arquivos css, um chamado home e outro global. O resultado final deverá ficar próximo a este:

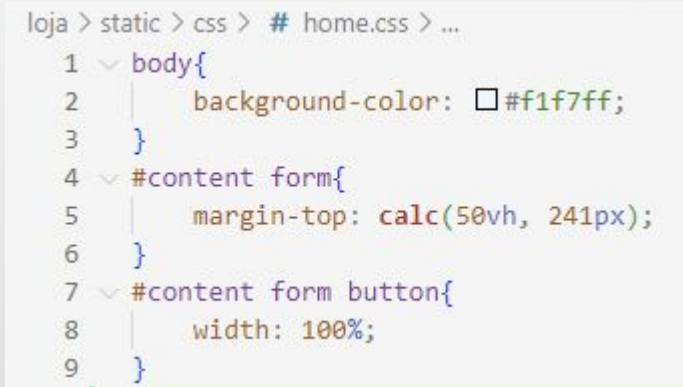


Django

Capítulo 7 - Trabalhando com Templates

Agora adicione o conteúdo do loja/static/css/**home.css**:

```
body{
    background-color: #f1f7ff;
}
#content form{
    margin-top: calc(50vh, 241px);
}
#content form button{
    width: 100%;
}
```

A screenshot of a code editor window showing the content of a file named 'home.css'. The file path is displayed at the top as 'loja > static > css > # home.css > ...'. The code is written in a syntax-highlighted style with line numbers on the left. The code defines styles for the body, a content form, and a content form button. The body has a background color of #f1f7ff. The content form has a margin-top of calc(50vh, 241px). The content form button has a width of 100%.

```
loja > static > css > # home.css > ...
1  < body{
2    |     background-color: #f1f7ff;
3    | }
4  < #content form{
5    |     margin-top: calc(50vh, 241px);
6    | }
7  < #content form button{
8    |     width: 100%;
9    | }
```


Django

Capítulo 7 - Trabalhando com Templates

Agora adicione o conteúdo do loja/static/css/**global.css**:

Parte 1:

```
#content{min-height: calc(100vh - 112px);}
.image-circle{width: 150px;height: 150px;border-radius: 100%;margin: 10px auto;
background-size: cover; background-repeat: no-repeat; background-position:
center;background-color: #ccc;}
.btn-card {width: 49.5%;float: left;font-size: 0.9em;}
.btn-card:last-child{margin-left: 1%;}
.specialties{padding: 0;list-style: none;min-height: 46px;}
.navigation{margin: 0 auto;}
.social-container{display: flex;}
.social-container span{display: flex;align-content: space-between;}
.social-container a {margin: 0 1%;width: 50%;height: 40px;line-height: 20px;}
.social-container a, .social-container a:hover, .social-container a:active,
.social-container a:visited {color: #fff}
```

Django

Capítulo 7 - Trabalhando com Templates

Agora adicione o conteúdo do loja/static/css/**global.css**:

Parte 2:

```
.social-container a:first-child {margin-left: 0;}  
.social-container a:last-child {margin-right: 0;}  
.social-container a#facebook, .social-container a#facebook:visited{background-color:  
#3b5999;}  
.social-container a#google, .social-container a#google:visited{background-color:  
#dd4b39;}  
.social-container a#facebook:hover, .social-container a#facebook:active{background-color:  
#383871;}  
.social-container a#google:hover, .social-container a#google:active{background-color:  
#9e3e2c;}
```

Django

Capítulo 7 - Trabalhando com Templates

Código completo do loja/static/css/global.css:

```
loja > static > css > # global.css > $ #content
```

```
1  #content{min-height: calc(100vh - 112px);}
2  .image-circle{width: 150px;height: 150px;border-radius: 100%;margin: 10px auto; background-size: cover; background-repeat: no-repeat; background-position: center;background-color: #ccc;}
3  .btn-card {width: 49.5%;float: left;font-size: 0.9em;}
4  .btn-card:last-child{margin-left: 1%;}
5  .specialties{padding: 0;list-style: none;min-height: 46px;}
6  .navigation{margin: 0 auto;}
7  .social-container{display: flex;}
8  .social-container span{display: flex;align-content: space-between;}
9  .social-container a {margin: 0 1%;width: 50%;height: 40px;line-height: 20px;}
10 .social-container a, .social-container a:hover, .social-container a:active, .social-container a:visited {color: #fff}
11 .social-container a:first-child {margin-left: 0;}
12 .social-container a:last-child {margin-right: 0;}
13 .social-container a#facebook, .social-container a#facebook:visited{background-color: #3b5999;}
14 .social-container a#google, .social-container a#google:visited{background-color: #dd4b39;}
15 .social-container a#facebook:hover, .social-container a#facebook:active{background-color: #383871;}
16 .social-container a#google:hover, .social-container a#google:active{background-color: #9e3e2c;}
```

Django

Capítulo 7 - Trabalhando com Templates

Agora ajuste a view Home(loja/views/HomeView.py) para ao invés de apresentar diretamente a mensagem Olá Mundo! acessar o arquivo de template Home.html criado.

Para isso pode substituir o import do HttpResponse para o render, Assim:

```
from django.shortcuts import render
```

E mudar o return para achar o template com o render:

```
return render(request, template_name='home/home.html', status=200)
```

Código completo:

```
from django.shortcuts import render  
def home_view(request):  
    return render(request, template_name='home/home.html', status=200)
```

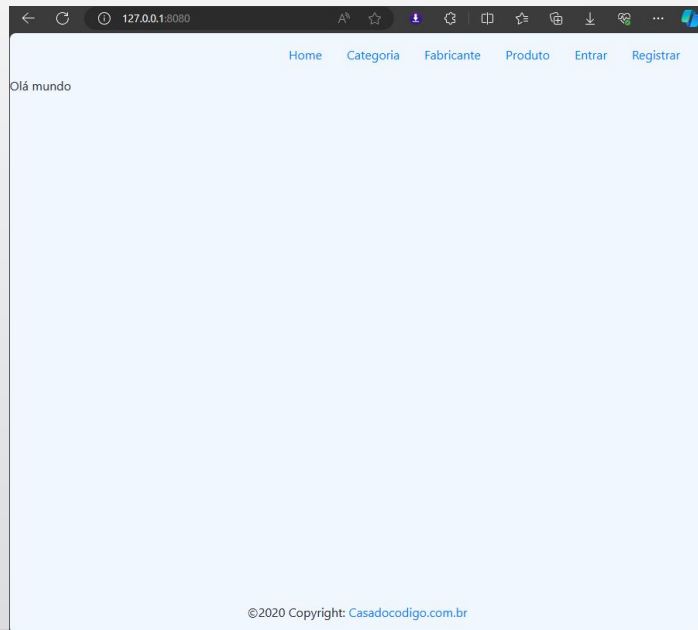
```
loja > views > HomeView.py > ...  
1  from django.shortcuts import render  
2  def home_view(request):  
3  |  return render(request, template_name='home/home.html', status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

após modificar a função da view Home e substituir o import, rode a aplicação e acesse o link: <http://127.0.0.1:8080>

```
C:\Users\Thiago Gonsalves>cd/loja  
C:\Loja>venv\Scripts\activate  
(venv) C:\Loja>python manage.py runserver 127.0.0.1:8080
```



Django

Capítulo 7 - Trabalhando com Templates

Vamos introduzir na página de entrada uma caixa de texto para busca de produtos. Para isso substitua o conteúdo do `{% block content %}{% endblock %}`, "`<div id='content'>Olá mundo</div>`", do `loja/templates/home/home.html` para o seguinte:

Parte 1 do conteúdo do `loja/templates/home/home.html`:

```
<div id="content">
    <div class="container">
        <div class="row">
            <div class="col-sm-12 col-md-6 col-lg-6 offset-sm-0">
                <form action="{% url 'home' %}" method="GET">
                    <h3>Buscar produtos</h3>
                    <div class="form-row">
                        <div class="col form-group">
                            <input type="text" class="form-control" name="produto"
placeholder="nome do produto">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do conteúdo do loja/templates/home/**home.html**:

```
</div>
    </div>
    <div class="form-row">
        <div class="col form-group mb-3">
            <button type="submit" class="btn btn-primary">Buscar</button>
        </div>
    </div>
</form>
</div></div></div></div>
```

Django

Capítulo 7 - Trabalhando com Templates

Código completo do loja/templates/home/**home.html**:

```
loja > templates > home > <> home.html > ...
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Inicio{% endblock %}
4  {% block styles %}
5  <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8  <div id="content">
9      <div class="container">
10         <div class="row">
11             <div class="col-sm-12 col-md-6 col-lg-6 offset-sm-0">
12                 <form action="{% url 'home' %}" method="GET">
13                     <h3>Buscar produtos</h3>
14                     <div class="form-row">
15                         <div class="col form-group">
16                             <input type="text" class="form-control" name="produto" placeholder="nome do produto">
17                         </div>
18                     </div>
19                     <div class="form-row">
20                         <div class="col form-group mb-3">
21                             <button type="submit" class="btn btn-primary">Buscar</button>
22                         </div>
23                     </div>
24                 </form>
25             </div>
26         </div>
27     </div>
28 </div>
29 {% endblock %}
30 {% block scripts %}
31 <script src="{% static 'js/home.js' %}"></script>
32 {% endblock %}
```


Django

Capítulo 7 - Trabalhando com Templates

A sintaxe do form action utilizada usa o nome da Url. Para tal, tem que ser incluído o nome da Url no arquivo de Urls. Ou seja, para utilizar a sintaxe abaixo no arquivo home.html

```
<form action={% url 'home' %} method="GET">
```

É necessário definir a Url usando a sintaxe abaixo no arquivo HomeUrls.py(loja/urls/HomeUrls.py) adicionando o parâmetro name.

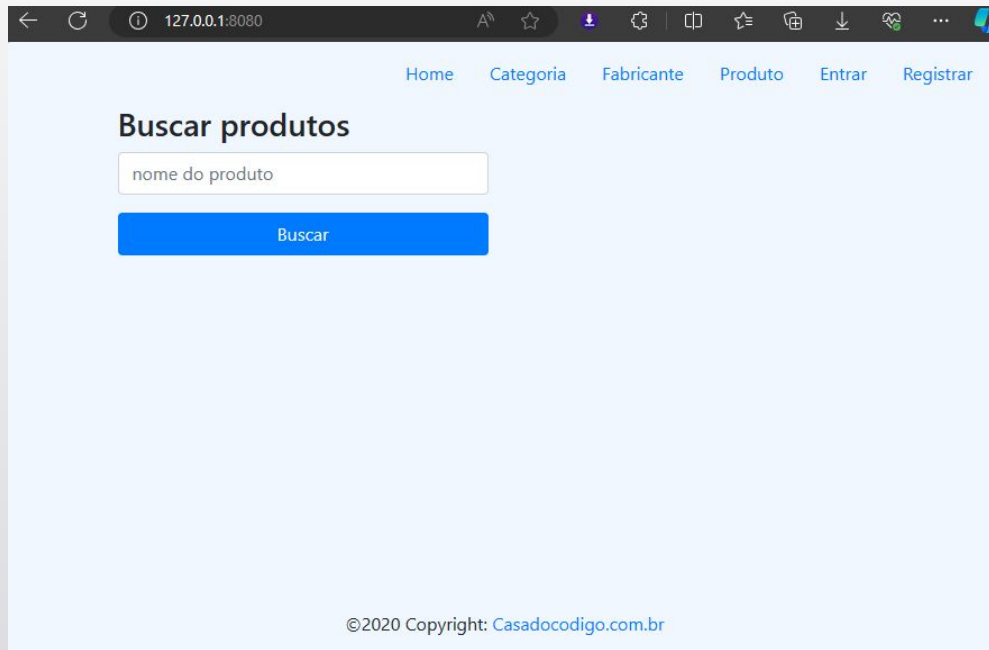
```
urlpatterns = [  
    path("", home_view, name= 'home'),  
]
```

```
loja > urls > HomeUrls.py > ...  
1  from django.urls import path  
2  from loja.views.HomeView import home_view  
3  
4  urlpatterns = [  
5      path("", home_view, name= 'home'),  
6  ]
```

Django

Capítulo 7 - Trabalhando com Templates

Agora rode novamente a aplicação e acesse o link: <http://127.0.0.1:8080> para conferir as alterações



Django

Capítulo 7 - Trabalhando com Templates

Agora vamos fazer com que ma view da Home(loja/views/HomeView.py) seja filtrado os produtos com base no nome e mostre eles na página do site. Primeiro importamos a classe modelo de produto assim:

```
from loja.models import Produto
```

Em seguida dentro da função home_view adicionaremos a lógica de filtragem com base no nome, assim:

```
produto = request.GET.get("produto")
produtos = Produto.objects.all()
if produto is not None:
    produtos = produtos.filter(Produto__contains=produto)
```

Django

Capítulo 7 - Trabalhando com Templates

Após adicionar a lógica, colocaremos, abaixo e fora do if, os resultados em formato de context para que possamos passar a lista de objetos filtrados para o template, assim:

```
context = {  
    'produtos': produtos  
}
```

Agora, para finalizar basta adicionar esse context como parâmetro do retorno da nossa função, assim:

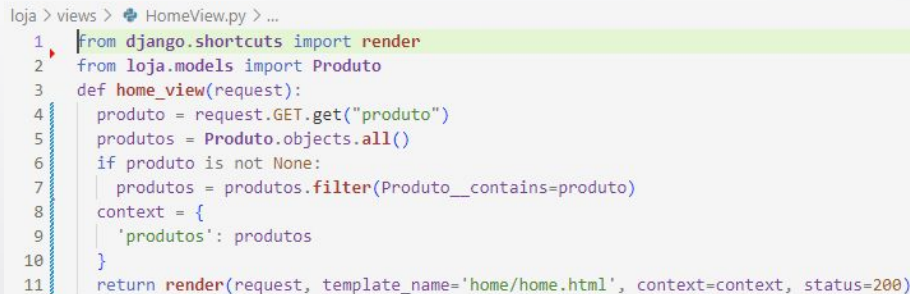
```
return render(request, template_name='home/home.html',  
context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Resultado final:

```
from django.shortcuts import render
from loja.models import Produto
def home_view(request):
    produto = request.GET.get("produto")
    produtos = Produto.objects.all()
    if produto is not None:
        produtos = produtos.filter(Produto__contains=produto)
    context = {
        'produtos': produtos
    }
    return render(request, template_name='home/home.html', context=context, status=200)
```



loja > views > HomeView.py > ...

```
1 from django.shortcuts import render
2 from loja.models import Produto
3 def home_view(request):
4     produto = request.GET.get("produto")
5     produtos = Produto.objects.all()
6     if produto is not None:
7         produtos = produtos.filter(Produto__contains=produto)
8     context = {
9         'produtos': produtos
10    }
11    return render(request, template_name='home/home.html', context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Agora vamos adicionar a lista filtrada na nossa página home.html. Para isso, no block content, coloque, abaixo do fechamento da div class="row", a seguinte estrutura html. Parte 1 da estrutura:

```
{% block content %}
<div id="content">
  <div class="container">
    <div class="row">
      <div class="col-sm-12 col-md-6 col-lg-6 offset-sm-0">
        <form action={% url 'home' %} method="GET">
          <h3>Buscar produtos</h3>
          <div class="form-row">
            <div class="col form-group">
              <input type="text" class="form-control" name="produto"
placeholder="nome do produto">
            </div>
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2:

```
</div><div class="form-row"><div class="col form-group mb-3"><button type="submit"
class="btn btn-primary">Buscar</button></div></div></form></div></div>
    <!--Adicione as linhas em destaque abaixo-->
    <div class="alert alert-info">Produtos encontrados [{{ produtos | length
}}]</div>
    <div class="row">
        {% for prod in produtos %}
        <div class="col-xs-9 col-md-3 col-lg-3 ">
            <div class="card mb-4">
                <div class="image-circle" style="background-image:
url('/media/{{prod.image}}');"></div>
                <div class="card-body">
                    <h5 class="card-title">{{prod.Produto}}</h5>
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3:

```
        <a href={% url 'home' %} class="btn btn-primary btn-card">
            <i class="fa fa-search"></i> Ver
        </a>
        <button class="btn btn-warning btn-card">
            <i class="fa fa-heart"></i> Favoritar</button>
    </div>
</div>
</div>
{% endfor %}
</div>
</div>
{% endblock %}
```


Django

Capítulo 7 - Trabalhando com Templates

```
{% block content %}
<div id="content">
  <div class="container">
    <div class="row">
      <div class="col-sm-12 col-md-6 col-lg-6 offset-sm-0">
        <form action={% url 'home' %} method="GET">
          <h3>Buscar produtos</h3>
          <div class="form-row">
            <div class="col form-group">
              <input type="text" class="form-control" name="produto" placeholder="nome do produto">
            </div>
          </div>
          <div class="form-row">
            <div class="col form-group mb-3">
              <button type="submit" class="btn btn-primary">Buscar</button>
            </div>
          </div>
        </form>
      </div>
    </div>
    <div class="alert alert-info">Produtos encontrados {{ produtos | length }}</div>
    <div class="row">
      {% for prod in produtos %}
        <div class="col-xs-9 col-md-3 col-lg-3 ">
          <div class="card mb-4">
            <div class="image-circle" style="background-image: url('/media/{{prod.image}}');"></div>
            <div class="card-body">
              <h5 class="card-title">{{prod.Produto}}</h5>
              <a href={% url 'home' %} class="btn btn-primary btn-card">
                <i class="fa fa-search"></i> Ver
              </a>
              <button class="btn btn-warning btn-card">
                <i class="fa fa-heart"></i> Favoritar</button>
            </div>
          </div>
        </div>
      {% endfor %}
    </div>
  </div>
</div>
{% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Explicando a sintaxe que faz parte do template:

`[{{ produtos | length }}]` esse é um comando para pegar o tamanho da lista passada no context(produtos é o nome que colocamos entre aspas simples).

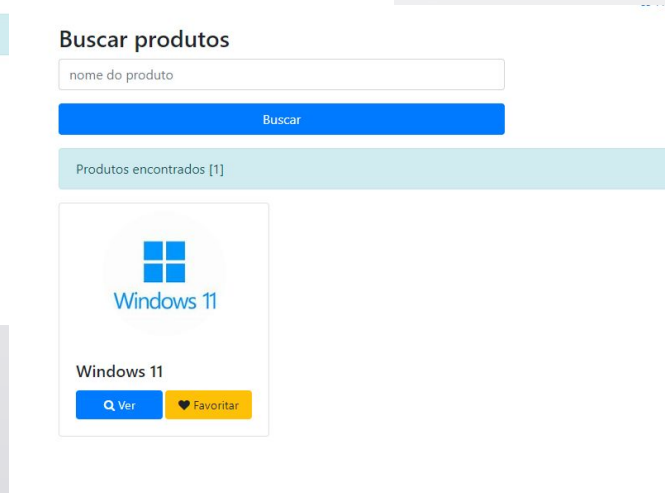
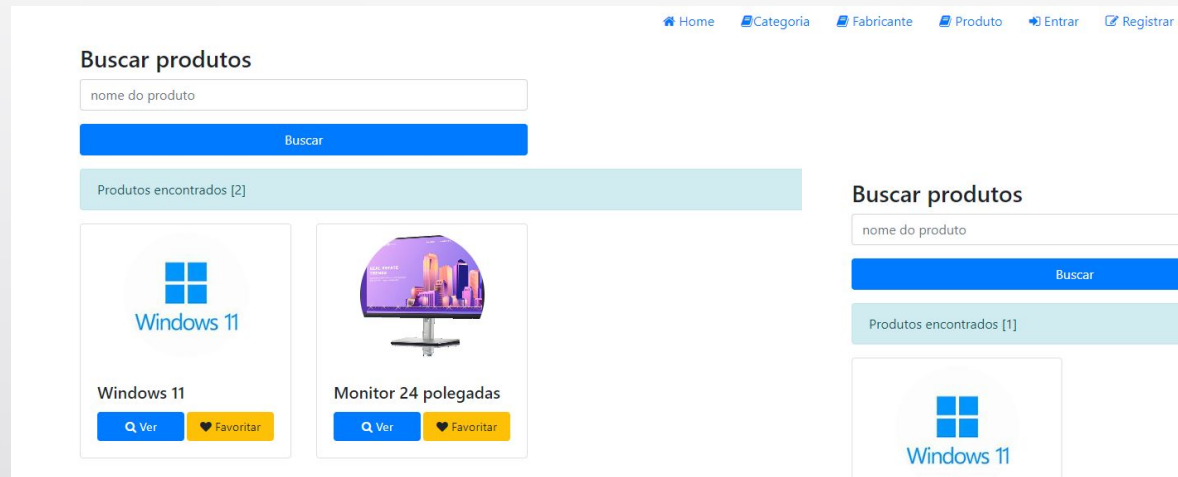
`{% for prod in produtos %}` esse comando percorre a lista e o `{% endfor %}` finaliza o loop

`{{prod.Produto}}` ou `{{prod.imaga}}` é a forma de acessar o atributo dos objetos acessados no for da lista, esse primeiro acessa o nome, e o segundo o nome da imagem

Django

Capítulo 7 - Trabalhando com Templates

Em seguida pode rodar a aplicação novamente e teste a pesquisa.

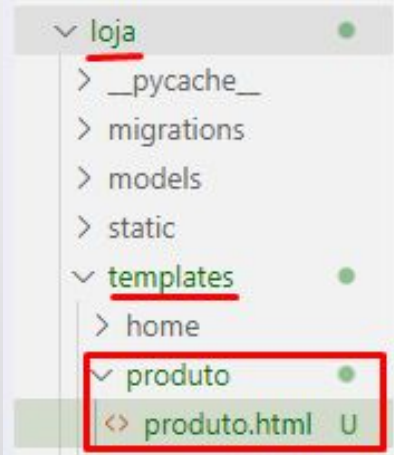


OBS: quando clicar em buscar o campo de pesquisa é limpo

Django

Capítulo 7 - Trabalhando com Templates

Agora iremos fazer a implementação do CRUD de produtos será. Iniciaremos com a apresentação da listagem de produtos de forma similar ao que foi feito no home. Crie dentro de templates uma pasta produto e dentro dela um arquivo produto.html, loja/templates/produto/**produto.html**



Django

Capítulo 7 - Trabalhando com Templates

Iremos fazer a listagem do crud semelhante a listagem do home, para isso Coloque o código abaixo no arquivo loja/templates/produto/**produto.html**. Parte 1 do código:

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
    <link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
<div id="content">
    <div class="container">
        <div class="alert alert-info">Produtos encontrados [{{ produtos |
length }}]</div>
        {% for prod in produtos %}
            <div class="row">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do código loja/templates/produto/**produto.html**:

```
<h5 class="col-md-3">{{prod.Produto}}</h5>
<a href={% url 'produto' prod.id %} class=
  "btn btn-primary mb-2 mr-2"><i class="fa fa-search"></i> Ver</a>
<a href={% url 'produto' prod.id %} class=
  "btn bg-secondary mb-2 mr-2"><i class="fa fa-edit"></i> Editar</a>
  <a href={% url 'produto' prod.id %} class="btn btn-warning mb-2">
<i class="fa fa-sign-in"></i> Apagar</a>
  </div>
{% endfor %}
</div>
{% endblock %}
{% block scripts %}
  <script src="{% static 'js/home.js' %}"></script>
{% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

código loja/templates/produto/**produto.html**:

```
loja > templates > produto > <> produto.html > ...
```

```
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Início{% endblock %}
4  {% block styles %}
5    <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8    <div id="content">
9      <div class="container">
10         <div class="alert alert-info">Produtos encontrados [{{ produtos | length }}]</div>
11         {% for prod in produtos %}
12           <div class="row">
13             <h5 class="col-md-3">{{prod.Produto}}</h5>
14             <a href="{% url 'details_produto' prod.id %}" class="btn btn-primary mb-2 mr-2"><i class="fa fa-search"></i> Ver</a>
15             <a href="{% url 'edit_produto' prod.id %}" class="btn bg-secondary mb-2 mr-2"><i class="fa fa-edit"></i> Editar</a>
16             <a href="{% url 'produto' prod.id %}" class="btn btn-warning mb-2"><i class="fa fa-sign-in"></i> Apagar</a>
17           </div>
18         {% endfor %}
19       </div>
20     </div>
21   {% endblock %}
22   {% block scripts %}
23     <script src="{% static 'js/home.js' %}"></script>
24   {% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

No arquivo `ProdutoView.py`, `loja/views/ProdutoView.html`, ajuste o método `list_produto_view` para carregar esse novo template. Apesar da interface não precisar ter todas as consultas da função, pode manter, comentar ou apagar o código existente sem problema. Adicione e mantenha o que está destacado para a listagem ser realizada. Parte 1 do código:

```
from django.shortcuts import render # Retire from django.http import HttpResponse
from loja.models import Produto
from datetime import timedelta, datetime
from django.utils import timezone

def list_produto_view(request, id=None):
    produto = request.GET.get("produto")
    destaque = request.GET.get("destaque")
    promocao = request.GET.get("promocao")
    categoria = request.GET.get("categoria")
```


Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do código loja/views/**ProdutoView.py**:

```
fabricante = request.GET.get("fabricante")
dias = request.GET.get("dias")
produtos = Produto.objects.all()
if produto is not None:
    produtos = produtos.filter(Produto__contains=produto)
if promocao is not None:
    produtos = produtos.filter(promocao=promocao)
if destaque is not None:
    produtos = produtos.filter(destaque=destaque)
if categoria is not None:
    produtos = produtos.filter(categoria__Categoria=categoria)
if fabricante is not None:
    produtos = produtos.filter(fabricante__Fabricante=fabricante)
if dias is not None:
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 do código loja/views/**ProdutoView.py**:

```
    now = timezone.now()
    now = now - timedelta(days = int(dias))
    produtos = produtos.filter(criado_em__gte=now)
    if id is not None:
        produtos = produtos.filter(id=id)
    print(produtos)
    # Adicione para definir o contexto e carregar o template
    context = {
        'produtos': produtos
    }
    return render(request, template_name='produto/produto.html',
context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

código loja/views/**ProdutoView.py**:

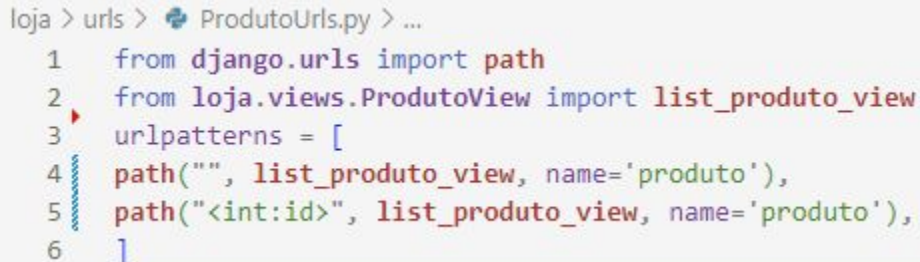
```
loja > views > ProdutoView.py > ...
1 from django.shortcuts import render # Retire from django.http import HttpResponseRedirect
2 from loja.models import Produto
3 from datetime import timedelta, datetime
4 from django.utils import timezone
5 def list_produto_view(request, id=None):
6     produto = request.GET.get("produto")
7     destaque = request.GET.get("destaque")
8     promocao = request.GET.get("promocao")
9     categoria = request.GET.get("categoria")
10    fabricante = request.GET.get("fabricante")
11    dias = request.GET.get("dias")
12    produtos = Produto.objects.all()
13    if produto is not None:
14        produtos = produtos.filter(Produto__contains=produto)
15    if promocao is not None:
16        produtos = produtos.filter(promocao=promocao)
17    if destaque is not None:
18        produtos = produtos.filter(destaque=destaque)
19    if categoria is not None:
20        produtos = produtos.filter(categoria__Categoria=categoria)
21    if fabricante is not None:
22        produtos = produtos.filter(fabricante__Fabricante=fabricante)
23    if dias is not None:
24        now = timezone.now()
25        now = now - timedelta(days = int(dias))
26        produtos = produtos.filter(criado_em_gte=now)
27    context = { 'produtos': produtos }
28    return render(request, template name='produto/produto.html', context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

No arquivo ProdutoUrls.py, loja/urls/**ProdutoUrls.py**, adicione o name para que as urls da função consigam ser localizadas.

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view
urlpatterns = [
    path("", list_produto_view, name= 'produto'),
    path("<int:id>", list_produto_view, name= 'produto'),
]
```



```
loja > urls > ProdutoUrls.py > ...
1  from django.urls import path
2  from loja.views.ProdutoView import list_produto_view
3  urlpatterns = [
4      path("", list_produto_view, name='produto'),
5      path("<int:id>", list_produto_view, name='produto'),
6  ]
```

Django

Capítulo 7 - Trabalhando com Templates

Para acessar o CRUD de produto, rode a aplicação acesse <http://127.0.0.1:8080/> e clique no link de produto na página principal. A página com a listagem de produtos aparece.



Django

Capítulo 7 - Trabalhando com Templates

Agora Será desenvolvida agora a interface de edição de produtos. Inicialmente será incluído na interface apenas 5 campos, são eles: id (identificador do produto), Produto (descrição do produto), destaque, promocao e msgPromocao (mensagem de promoção). Adicione na pasta templates/produto o arquivo produto-edit.html que apresentará a visualização do produto para edição. Inclua o seguinte código no arquivo loja/templates/produto/**produto-edit.html**:

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
<link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
<div id="content">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do código do loja/templates/produto/**produto-edit.html**:

```
<div class="container">
<form method="POST" action= "/produto/edit" >
{% csrf_token %}
<div class="row">
<input type="hidden" value="{{produto.id}}" name="id">
<h5 for="Produto" class="col-md-2">Produto</h5>
<input value="{{produto.Produto}}" name="Produto">
</div>
<div class="row">
<div class="col-md-2">
{% if produto.destaque %}
<input type="checkbox" checked name="destaque" >
{% else %}
<input type="checkbox" name="destaque" >
{% endif %}
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 do código do loja/templates/produto/**produto-edit.html**:

```
<label for="destaque">Destaque</label>
</div>
</div>
<div class="row">
<div class="col-md-2">
{% if produto.promocao %}
<input type="checkbox" checked name="promocao" >
{% else %}
<input type="checkbox" name="promocao" >
{% endif %}
<label for="promocao">Promoção</label>
</div>
</div>
<div class="row">
<label for="msgPromocao" class="col-md-2">Msg Promoção</label>
```


Django

Capítulo 7 - Trabalhando com Templates

Parte 4 do código do loja/templates/produto/**produto-edit.html**:

```
<input value="{{produto.msgPromocao}}" name="msgPromocao">
</div>
<div class="row">
<button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i class="fa
fa-check"></i> Salvar</button>
<a href={% url 'produto' %} class="btn bg-secondary col-md-2 mt-3"><i class="fa
fa-rotate-left"></i> Retornar</a>
</div>
</form>
</div>
</div>
{% endblock %}
{% block scripts %}
<script src="{% static 'js/home.js' %}"></script>
{% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

código do

loja/templates/produto/**produto-edit.html**:

```
loja > templates > produto > <> produto-edit.html > ...
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Início{% endblock %}
4  {% block styles %}
5    <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8    <div id="content">
9      <div class="container">
10        <form method="POST" action="/produto/edit">
11          {% csrf_token %}
12          <div class="row">
13            <input type="hidden" value="{{produto.id}}" name="id">
14            <h5 for="Produto" class="col-md-2">Produto</h5>
15            <input value="{{produto.Produto}}" name="Produto">
16          </div>
17          <div class="row">
18            <div class="col-md-2">
19              {% if produto.destaque %}
20                <input type="checkbox" checked name="destaque">
21              {% else %}
22                <input type="checkbox" name="destaque">
23              {% endif %}
24              <label for="destaque">Destaque</label>
25            </div>
26          </div>
27          <div class="row">
28            <div class="col-md-2">
29              {% if produto.promocao %}
30                <input type="checkbox" checked name="promocao">
31              {% else %}
32                <input type="checkbox" name="promocao">
33              {% endif %}
34              <label for="promocao">Promoção</label>
35            </div>
36          </div>
37          <div class="row">
38            <label for="msgPromocao" class="col-md-2">Msg Promoção</label>
39            <input value="{{produto.msgPromocao}}" name="msgPromocao">
40          </div>
41          <div class="row">
42            <button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i class="fa fa-check"></i>Salvar</button>
43            <a href="{% url 'produto' %}" class="btn bg-secondary col-md-2 mt-3"><i class="fa fa-rotate-left"></i>Retornar</a>
44          </div>
45        </form>
46      </div>
47    </div>
48  {% endblock %}
49  {% block scripts %}
50    <script src="{% static 'js/home.js' %}"></script>
51  {% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Os ícones usados no nosso projeto de coração, lupa, editar, e outros, vem do site <https://fontawesome.com/search?o=r&m=free&s=solid&f=classic> que utilizamos no loja/templates/**base.html** no link com href sendo

No exemplo, o botão de salvar usa um fa-check e o botão de retornar fa-rotate-left.

```
<div class="row">
  <button type="submit" class="btn btn-danger col-md-2 mt-3mr-2"><i class="fa
fa-check"></i> Salvar</button>
  <a href={% url 'produto' %} class="btn bg-secondary col-md-2 mt-3"><i class="fa
fa-rotate-left"></i> Retornar</a>
</div>
```

Django

Capítulo 7 - Trabalhando com Templates

Agora nas funções da view de produto terá uma diferença em relação a listagem, usaremos dois métodos, um apenas para montar a interface(função GET) e outro para modificar o banco(função POST). Primeiro vamos adicionar na view de produto a função que irá carregar os dados do produto a ser editado. Adicione em loja/views/**ProdutoView.py** o código abaixo:

```
def edit_produto_view(request, id=None):
```

```
    produtos = Produto.objects.all()
```

```
    if id is not None:
```

```
        produtos = produtos.filter(id=id)
```

```
    produto = produtos.first()
```

```
    print(produto)
```

```
    context = { 'produto': produto }
```

```
    return render(request, template_name='produto/produto-edit.html', context=context,  
status=200)
```



```
loja > views > ProdutoView.py > ...  
1 from django.shortcuts import render # Retire from django.http import HttpResponseRedirect  
2 from loja.models import Produto  
3 from datetime import timedelta, datetime  
4 from django.utils import timezone  
5  
6 def edit_produto_view(request, id=None):  
7     produtos = Produto.objects.all()  
8     if id is not None:  
9         produtos = produtos.filter(id=id)  
10    produto = produtos.first()  
11    print(produto)  
12    context = { 'produto': produto }  
    return render(request, template_name='produto/produto-edit.html', context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

No arquivo `ProdutoUrls.py` adicione os novos path para essa nova função criada. Adicione o trechos destacados abaixo no arquivo `loja/urls/ProdutoUrls.py`:

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view, edit_produto_view
urlpatterns = [
    path("", list_produto_view, name= 'produto'),
    path("<int:id>", list_produto_view, name= 'produto'),
    path("edit/<int:id>", edit_produto_view, name= 'edit_produto'),
]
```



```
loja > urls > ProdutoUrls.py > ...
1  from django.urls import path
2  from loja.views.ProdutoView import list_produto_view, edit_produto_view
3  urlpatterns = [
4      path("", list_produto_view, name= 'produto'),
5      path("<int:id>", list_produto_view, name= 'produto'),
6      path("edit/<int:id>", edit_produto_view, name= 'edit_produto'),
7  ]
```

Django

Capítulo 7 - Trabalhando com Templates

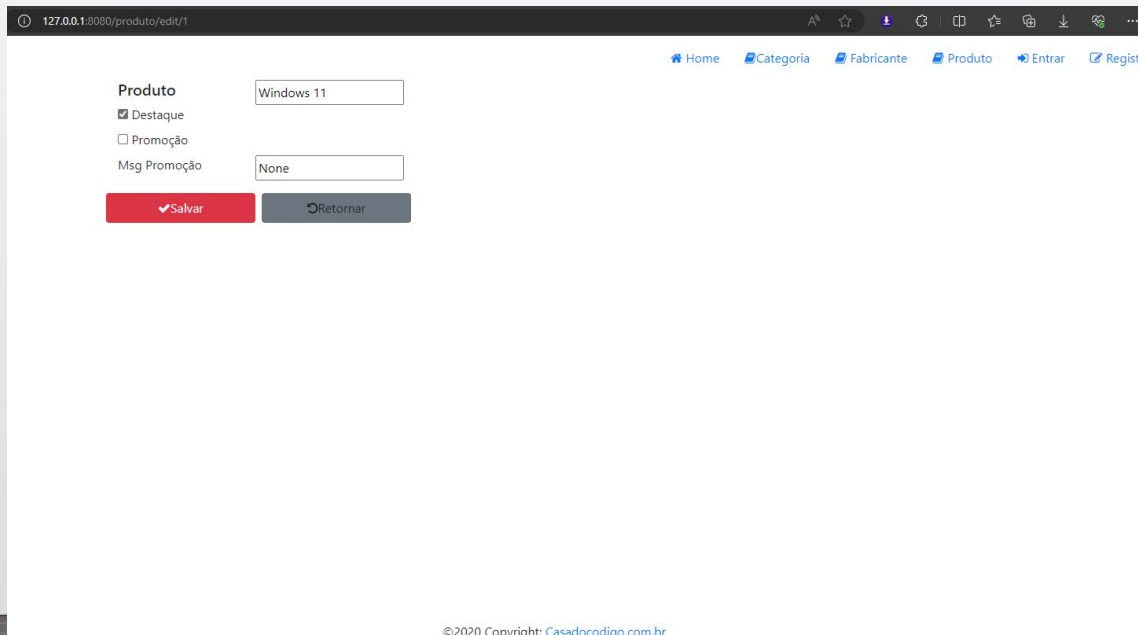
Por fim volte no `html loja/templates/produto/produto.html` e modifique na âncora do botão editar o url 'produto' para o mesmo valor do name da url da pagina editar('edit_produto') do path associado a função GET da view do editar(edit_produto_view)

```
{% block content %}
<div id="content">
  <div class="container">
    <div class="alert alert-info">Produtos encontrados [{{ produtos | length }}]</div>
    {% for prod in produtos %}
      <div class="row">
        <h5 class="col-md-3">{{ prod.Produto }}</h5>
        <a href="{% url 'produto' prod.id %}" class="btn btn-primary mb-2 mr-2"><i class="fa fa-search"></i> Ver</a>
        <a href="{% url 'edit_produto' prod.id %}" class="btn bg-secondary mb-2 mr-2"><i class="fa fa-edit"></i> Editar</a>
        <a href="{% url 'produto' prod.id %}" class="btn btn-warning mb-2"><i class="fa fa-sign-in"></i> Apagar</a>
      </div>
    {% endfor %}
  </div>
</div>
{% endblock %}
{% block scripts %}
```

Django

Capítulo 7 - Trabalhando com Templates

Execute a aplicação, vá na listagem do crud e clique no botão editar para ver a nova interface.



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8080/produto/edit/1'. The page features a navigation menu at the top with links: Home, Categoria, Fabricante, Produto, Entrar, and Regist. The main content area is titled 'Produto' and contains a form with the following fields and controls:

- A text input field containing 'Windows 11'.
- A checkbox labeled 'Destaque' which is checked.
- A checkbox labeled 'Promoção' which is unchecked.
- A text input field labeled 'Msg Promoção' containing 'None'.
- A red button labeled 'Salvar' with a checkmark icon.
- A grey button labeled 'Retornar' with a circular arrow icon.

At the bottom of the page, there is a copyright notice: '©2020 Copyright: Casadocodigo.com.br'.

Django

Capítulo 7 - Trabalhando com Templates

Para que o botão de salvar funcione, precisamos adicionar a programação do evento POST na função de edição. Adicione no arquivo loja/views/**ProdutoView.py** a funcionalidade de edição. Ela será implementada em uma nova função denominada `edit_produto_postback` que será disparada ao clicar o botão de submit da interface de edição.

inclua o acesso ao redirect

```
from django.shortcuts import render, redirect
```

```
from loja.models import Produto
```

```
from datetime import timedelta, datetime
```

```
from django.utils import timezone
```

```
def list_produto_view(request, id=None): [...]
```

```
def edit_produto_view(request, id=None): [...]
```

adicione a função que trata o postback da interface de edição

```
def edit_produto_postback(request, id=None):
```

```
    # Processa o post back gerado pela action
```


Django

Capítulo 7 - Trabalhando com Templates

parte 2 do código do loja/views/**ProdutoView.py**:

```
if request.method == 'POST':  
    # Salva dados editados  
    id = request.POST.get("id")  
    produto = request.POST.get("Produto")  
    destaque = request.POST.get("destaque")  
    promocao = request.POST.get("promocao")  
    msgPromocao = request.POST.get("msgPromocao")  
    print("postback")  
    print(id)  
    print(produto)  
    print(destaque)  
    print(promocao)  
    print(msgPromocao)  
    try:
```

Django

Capítulo 7 - Trabalhando com Templates

parte 3 do código do loja/views/**ProdutoView.py**:

```
    obj_produto = Produto.objects.filter(id=id).first()
    obj_produto.Produto = produto
    obj_produto.destaque = (destaque is not None)
    obj_produto.promocao = (promocao is not None)
    if msgPromocao is not None:
        obj_produto.msgPromocao = msgPromocao
    obj_produto.save()
    print("Produto %s salvo com sucesso" % produto)
except Exception as e:
    print("Erro salvando edição de produto: %s" % e)

return redirect("/produto")
```

Django

Capítulo 7 - Trabalhando com Templates

Código do loja/views/**ProdutoView.py**:

```
loja > views > ProdutoView.py > ...
1 from django.shortcuts import render, redirect
2 from loja.models import Produto
3 from datetime import timedelta, datetime
4 from django.utils import timezone
5 # adicione a função que trata o postback da interface de edição
6 def edit_produto_postback(request, id=None):
7     # Processa o post back gerado pela action
8     if request.method == 'POST':
9         # Salva dados editados
10         id = request.POST.get("id")
11         produto = request.POST.get("Produto")
12         destaque = request.POST.get("destaque")
13         promocao = request.POST.get("promocao")
14         msgPromocao = request.POST.get("msgPromocao")
15         print("postback")
16         print(id)
17         print(produto)
18         print(destaque)
19         print(promocao)
20         print(msgPromocao)
21         try:
22             obj_produto = Produto.objects.filter(id=id).first()
23             obj_produto.Produto = produto
24             obj_produto.destaque = (destaque is not None)
25             obj_produto.promocao = (promocao is not None)
26             if msgPromocao is not None:
27                 obj_produto.msgPromocao = msgPromocao
28             obj_produto.save()
29             print("Produto %s salvo com sucesso" % produto)
30         except Exception as e:
31             print("Erro salvando edição de produto: %s" % e)
32         return redirect("/produto")
```

Django

Capítulo 7 - Trabalhando com Templates

No arquivo loja/urls/**ProdutoUrls.py** inclua o acesso a função do postback:

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view, edit_produto_view,
edit_produto_postback
urlpatterns = [
    path("", list_produto_view, name= 'produto'),
    path("<int:id>", list_produto_view, name= 'produto'),
    path("edit/<int:id>", edit_produto_view, name= 'edit_produto'),
    path("edit", edit_produto_postback, name= 'edit_produto_postback'),
]
```

loja > urls > ProdutoUrls.py > ...

```
1  from django.urls import path
2  from loja.views.ProdutoView import list_produto_view, edit_produto_view, edit_produto_postback
3  urlpatterns = [
4      path("", list_produto_view, name= 'produto'),
5      path("<int:id>", list_produto_view, name= 'produto'),
6      path("edit/<int:id>", edit_produto_view, name= 'edit_produto'),
7      path("edit", edit_produto_postback, name= 'edit_produto_postback'),
8  ]
```

Django

Capítulo 7 - Trabalhando com Templates

Acessa a página e faça a edição de um dos produtos:

Produto

☒ Destaque

☒ Promoção

Msg Promoção

✓ Salvar

↶ Retornar

Produtos encontrados [2]		
Windows	Ver	Editar Apagar
Monitor 24 polegadas	Ver	Editar Apagar

Django

Capítulo 7 - Trabalhando com Templates

Agora será realizado a operação de Ver. Adicione no arquivo `ProdutoView.py` a funcionalidade de visualização de detalhes do produto, primeiro crie o método `detais_produto_view`. A implementação dessa função é similar a `edit_produto_view` pois ela localiza e carrega o produto e passa para o template. Adicione no final do arquivo `loja/views/ProdutoView.py` a seguinte função para isso:

```
def details_produto_view(request, id=None):
    # Processa o evento GET gerado pela action
    produtos = Produto.objects.all()
    if id is not None:
        produtos = produtos.filter(id=id)
    produto = produtos.first()
    print(produto)
    context = {'produto': produto}
    return render(request, template_name='produto/produto-details.html', context=context,
status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Código do arquivo loja/views/**ProdutoView.py**:

loja > views >  ProdutoView.py > ...

```
1  from django.shortcuts import render, redirect
2  from loja.models import Produto
3  from datetime import timedelta, datetime
4  from django.utils import timezone
5  ✓ def details_produto_view(request, id=None):
6      # Processa o evento GET gerado pela action
7      produtos = Produto.objects.all()
8  ✓  if id is not None:
9      |   produtos = produtos.filter(id=id)
10     produto = produtos.first()
11     print(produto)
12     context = {'produto': produto}
13     return render(request, template_name='produto/produto-details.html', context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Agora é preciso criar na pasta template de produto o arquivo html que visualiza todos os detalhes do produto. Por simplificação iremos apenas copiar a interface utilizada na interface de edição e acrescentar a imagem do produto. Alguns campos dos produtos ainda não incluídos serão adicionados posteriormente.

Faça uma cópia do arquivo produto-edit.html com nome **produto-details.html** e faça os seguintes ajustes:

- (1) retire a TAG form pois a interface não vai submeter dados de retorno;
- (2) Para manter a visualização similar, deixe os inputs, mas acrescente a opção disabled de forma que o usuário não possa usar o campo para edição e
- (3) Retire o botão de Salvar.

Django

Capítulo 7 - Trabalhando com Templates

O conteúdo html do loja/templates/produto/**produto-details.html** deve ficar como o código apresentado a seguir. As alterações citadas no slide anterior estão em destaque.

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
    <link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
<div id="content">
    <div class="container">
        <!-- <form method="POST" action= "/produto/edit" >
            {% csrf_token %}-->
            <div class="row">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do código html do loja/templates/produto/**produto-details.html**:

```
<!--<input type="hidden" value="{{produto.id}}" name="id">-->
<h5 for="Produto" class="col-md-2">Produto</h5>
<input value="{{produto.Produto}}" name="Produto" disabled>
</div>
<div class="row">
<div class="col-md-2">
{% if produto.destaque %}
    <input type="checkbox" checked name="destaque" disabled>
{% else %}
    <input type="checkbox" name="destaque" disabled>
{% endif %}
<label for="destaque">Destaque</label>
</div>
</div>
<div class="row">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 do código html do loja/templates/produto/**produto-details.html**:

```
<div class="col-md-2">
    {% if produto.promocao %}
        <input type="checkbox" checked name="promocao" disabled>
    {% else %}
        <input type="checkbox" name="promocao" disabled>
    {% endif %}
<label for="promocao">Promoção</label>
</div>
</div>
<div class="row">
<label for="msgPromocao" class="col-md-2">Msg Promoção</label>
<input value="{{produto.msgPromocao}}" name="msgPromocao" disabled>
</div>
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 4 do código html do loja/templates/produto/**produto-details.html**:

```
<div class="row mt-3">
    
</div>
<div class="row">
    <!--<button type="submit" class="btn btn-danger col-md-2 mt-3
mr-2"><i class="fa fa-check"></i> Salvar</button>-->
    <a href={% url 'produto' %} class="btn bg-secondary col-md-2
mt-3"><i class="fa fa-rotate-left"></i> Retornar</a>
</div>
<!--</form>-->
</div>
</div>
{% endblock %}{% block scripts %}<script src="{% static 'js/home.js' %}"></script>{%
endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

código html completo do loja/templates/produto/**produto-details.html**:

```
loja > templates > produto > < produto-details.html > ...
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Inicio{% endblock %}
4  {% block styles %}
5  <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8  <div id="content">
9    <div class="container">
10     ...form method="post" action="/produto/edit/" ...
11     <!--{{form.as_table}}-->
12     <div class="row">
13       <!--<input type="hidden" value="{{produto.id}}" name="id"-->
14       <h3 form="Produto" class="col-md-2">Produto</h3>
15       <input value="{{produto.Produto}}" name="Produto" disabled>
16     </div>
17     <div class="row">
18       <div class="col-md-2">
19         {% if produto.destaque %}
20         <input type="checkbox" checked name="destaque" disabled>
21         {% else %}
22         <input type="checkbox" name="destaque" disabled>
23         {% endif %}
24         <label form="destaque">Destaque</label>
25       </div>
26     </div>
27     <div class="row">
28       <div class="col-md-2">
29         {% if produto.promocao %}
30         <input type="checkbox" checked name="promocao" disabled>
31         {% else %}
32         <input type="checkbox" name="promocao" disabled>
33         {% endif %}
34         <label form="promocao">Promoção</label>
35       </div>
36     </div>
37     <div class="row">
38       <label form="msgPromocao" class="col-md-2">Msg Promoção</label>
39       <input value="{{produto.msgPromocao}}" name="msgPromocao" disabled>
40     </div>
41     <div class="row mt-3">
42       
43     </div>
44     <div class="row">
45       <!--<button type="submit" class="btn btn-danger col-md-2 mt-3" -->
46       <div class="row">
47         <div class="col">
48           <a href="{% url 'produto' %}" class="btn btn-secondary col-md-2 mt-3">
49             <i class="fa fa-rotate-left"></i> Retornar</a>
50         </div>
51       </div>
52     </div>
53   {% endblock %}
54   {% block scripts %}
55   <script src="{% static 'js/home.js' %}"></script>
56   {% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Agora inclua no arquivo de urls, loja/urls/**ProdutoUrls.py**, o leiaute da chamada para a interface de detalhe do produto e o import da view.

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view, edit_produto_view,
edit_produto_postback, details_produto_view
urlpatterns = [
    path("", list_produto_view, name= 'produto'),
    path("<int:id>", list_produto_view, name= 'produto'),
    path("edit/<int:id>", edit_produto_view, name= 'edit_produto'),
    path("edit", edit_produto_postback, name= 'edit_produto_postback'),
    path("details/<int:id>", details_produto_view, name= 'details_produto'),
]
```

Django

Capítulo 7 - Trabalhando com Templates

E no template loja/templates/produto/**produto.html** altere a url a ser chamada no botão de Ver para 'details_produto', como no exemplo a seguir:

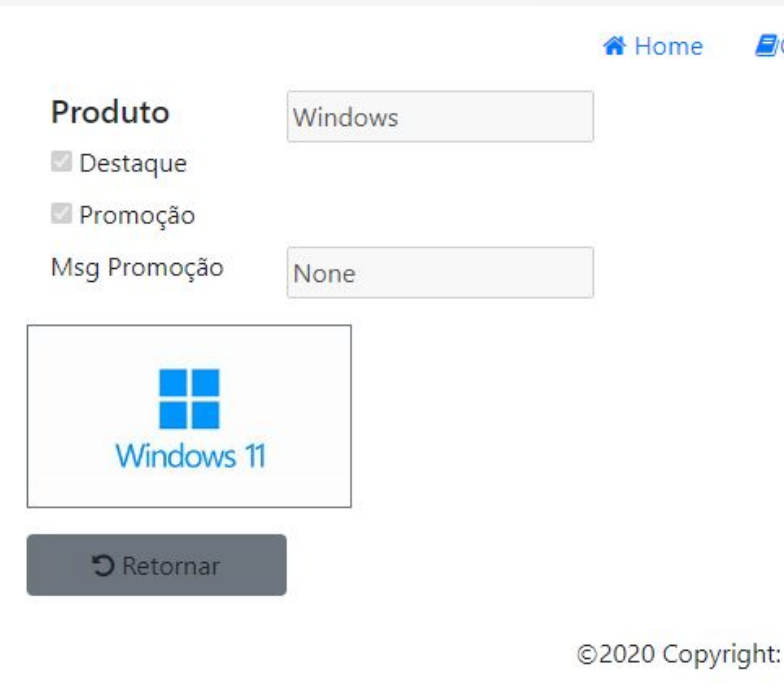
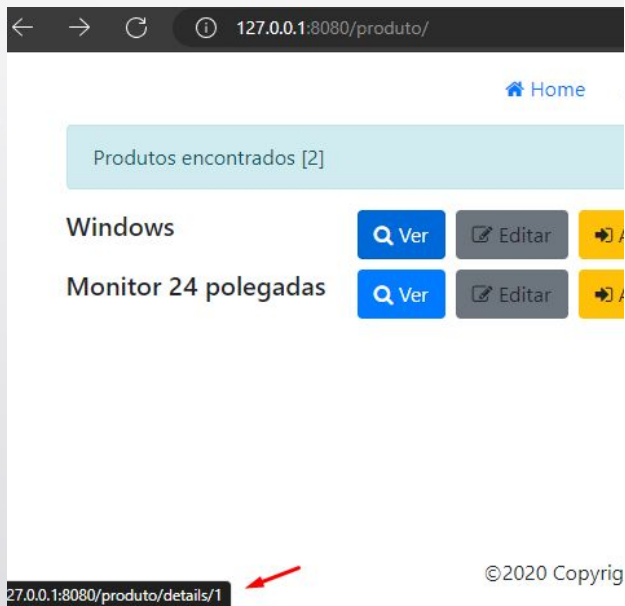
```
{% extends "base.html" %}
[...]
{% for prod in produtos %}
    <div class="row">
        <h5 class="col-md-3">{{prod.Produto}}</h5>
        <a href={% url 'details_produto' prod.id %} class="btn btn-primary
mb-2 mr-2"><i class="fa fa-search"></i> Ver</a>
        <a href={% url 'edit_produto' prod.id %} class="btn bg-secondary
mb-2 mr-2"><i class="fa fa-edit"></i> Editar</a>
        <a href={% url 'produto' prod.id %} class="btn btn-warning mb-2"
><i class="fa fa-trash"></i> Apagar</a>
    </div>
{% endfor %}
[...]
```

Django

Capítulo 7 - Trabalhando com Templates

Agora rode a aplicação e acione a funcionalidade editar

Observe a mudança na URL e a tela de detalhe de produto



Django

Capítulo 7 - Trabalhando com Templates

Continuando agora será realizado a operação do botão de Excluir que deve abrir uma página para mostrar todos os detalhes do produto. Adicione no arquivo `loja/views/ProdutoView.py` a funcionalidade de visualização do produto a ser excluído de forma que o usuário confira o que ele vai excluir. Ela será implementada em uma nova função denominada `delete_produto_view` que será disparada ao clicar o botão de Excluir da interface. A implementação dessa função é similar a `edit_produto_view` pois ela localiza e carrega o produto e passa para o template `produto-delete.html` que será criado. Adicione no final do arquivo a função a seguir:

```
def delete_produto_view(request, id=None):  
    # Processa o evento GET gerado pela action  
    produtos = Produto.objects.all()  
    if id is not None:  
        produtos = produtos.filter(id=id)  
    produto = produtos.first()
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 da função `delete_produto_view` do arquivo `loja/views/ProdutoView.py`:

```
print(produto)
context = {'produto': produto}
return render(request, template_name='produto/produto-delete.html', context=context,
status=200)
```

loja > views >  ProdutoView.py > ...

```
1  from django.shortcuts import render, redirect
2  from loja.models import Produto
3  from datetime import timedelta, datetime
4  from django.utils import timezone
5  def delete_produto_view(request, id=None):
6      # Processa o evento GET gerado pela action
7      produtos = Produto.objects.all()
8      if id is not None:
9          produtos = produtos.filter(id=id)
10     produto = produtos.first()
11     print(produto)
12     context = {'produto': produto}
13     return render(request, template_name='produto/produto-delete.html', context=context, status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Agora é preciso criar na pasta template de produto o arquivo html, produto-delete.html , que visualiza o produto a ser excluído para que o usuário possa conferir sua exclusão. Nessa interface haverá o botão de excluir para realizar a operação. Por simplificação iremos apenas copiar a interface utilizada na interface de edição e ajustar a funcionalidade do botão.

Faça uma cópia do arquivo produto-edit.html com nome produto-delete.html e faça os seguintes ajustes:

- (1) acrescente a opção disabled nos inputs para ter uma apresentação similar as demais interfaces
- (2) ajuste a action do form para `"/produto/delete"`
- (3) substitua o texto do botão de Salvar para Excluir. As alterações estão destacadas no código a seguir.

Django

Capítulo 7 - Trabalhando com Templates

O conteúdo html do loja/templates/produto/**produto-delete.html** deve ficar como o código apresentado a seguir. As alterações citadas no slide anterior estão em destaque.

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
    <link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
<div id="content">
    <div class="container">
        <form method="POST" action= "/produto/delete" >
            {% csrf_token %}
            <div class="row">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do conteúdo html do loja/templates/produto/**produto-delete.html**:

```
<input type="hidden" value="{{produto.id}}" name="id">
<h5 for="Produto" class="col-md-2">Produto</h5>
<input value="{{produto.Produto}}" name="Produto" disabled>
</div>
<div class="row">
<div class="col-md-2">
{% if produto.destaque %}
    <input type="checkbox" checked name="destaque" disabled>
{% else %}
    <input type="checkbox" name="destaque" disabled>
{% endif %}
<label for="destaque">Destaque</label>
</div>
</div>
<div class="row">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 do conteúdo html do loja/templates/produto/**produto-delete.html**:

```
<div class="col-md-2">
  {% if produto.promocao %}
    <input type="checkbox" checked name="promocao" disabled>
  {% else %}
    <input type="checkbox" name="promocao" disabled>
  {% endif %}
  <label for="promocao">Promoção</label>
</div>
</div>
<div class="row">
  <label for="msgPromocao" class="col-md-2">Msg Promoção</label>
  <input value="{{produto.msgPromocao}}" name="msgPromocao" disabled>
</div>
<div class="row">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 4 do conteúdo html do loja/templates/produto/**produto-delete.html**:

```
<button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i class="fa  
fa-check"></i> Excluir</button>  
    <a href={% url 'produto' %} class="btn bg-secondary col-md-2 mt-3"><i  
class="fa fa-rotate-left"></i> Retornar</a>  
    </div>  
    </form>  
    </div>  
</div>  
{% endblock %}  
{% block scripts %}  
    <script src="{% static 'js/home.js' %}"></script>  
{% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

O conteúdo html do loja/templates/produto/**produto-delete.html**:

```
loja > templates > produto > < produto-delete.html > ...
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Inicio{% endblock %}
4  {% block styles %}
5  <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8  <div id="content">
9      <div class="container">
10         <form method="POST" action="/produto/delete">
11             {% csrf_token %}
12             <div class="row">
13                 <input type="hidden" value="{{{produto.id}}}" name="id">
14                 <h5 for="Produto" class="col-md-2">Produto</h5>
15                 <input value="{{{produto.Produto}}}" name="Produto" disabled>
16             </div>
17             <div class="row">
18                 <div class="col-md-2">
19                     {% if produto.destaque %}
20                     <input type="checkbox" checked name="destaque" disabled>
21                     {% else %}
22                     <input type="checkbox" name="destaque" disabled>
23                     {% endif %}
24                     <label for="destaque">Destaque</label>
25                 </div>
26             </div>
27             <div class="row">
28                 <div class="col-md-2">
29                     {% if produto.promocao %}
30                     <input type="checkbox" checked name="promocao" disabled>
31                     {% else %}
32                     <input type="checkbox" name="promocao" disabled>
33                     {% endif %}
34                     <label for="promocao">Promoção</label>
35                 </div>
36             </div>
37             <div class="row">
38                 <label for="msgPromocao" class="col-md-2">Msg Promoção</label>
39                 <input value="{{{produto.msgPromocao}}}" name="msgPromocao" disabled>
40             </div>
41             <div class="row">
42                 <button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i class="fa fa-check"></i> Excluir</button>
43                 <a href="{% url 'produto' %}" class="btn bg-secondary col-md-2 mt-3"><i class="fa fa-rotate-left"></i> Retornar</a>
44             </div>
45         </form>
46     </div>
47 </div>
48 {% endblock %}
49 {% block scripts %}
50 <script src="{% static 'js/home.js' %}"></script>
51 {% endblock %}
```


Django

Capítulo 7 - Trabalhando com Templates

Agora inclua no arquivo de urls, loja/urls/**ProdutoUrls.py**, o leiaute da chamada para a interface de excluir produto. Adicione o código destacado abaixo:

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view, edit_produto_view,
edit_produto_postback, details_produto_view, delete_produto_view
urlpatterns = [
    path("", list_produto_view, name='produto'),
    path("<int:id>", list_produto_view, name='produto'),
    path("edit/<int:id>", edit_produto_view, name='edit_produto'),
    path("edit", edit_produto_postback, name='edit_produto_postback'),
    path("details/<int:id>", details_produto_view, name='details_produto'),
    path("delete/<int:id>", delete_produto_view, name='delete_produto'),
]
```

Django

Capítulo 7 - Trabalhando com Templates

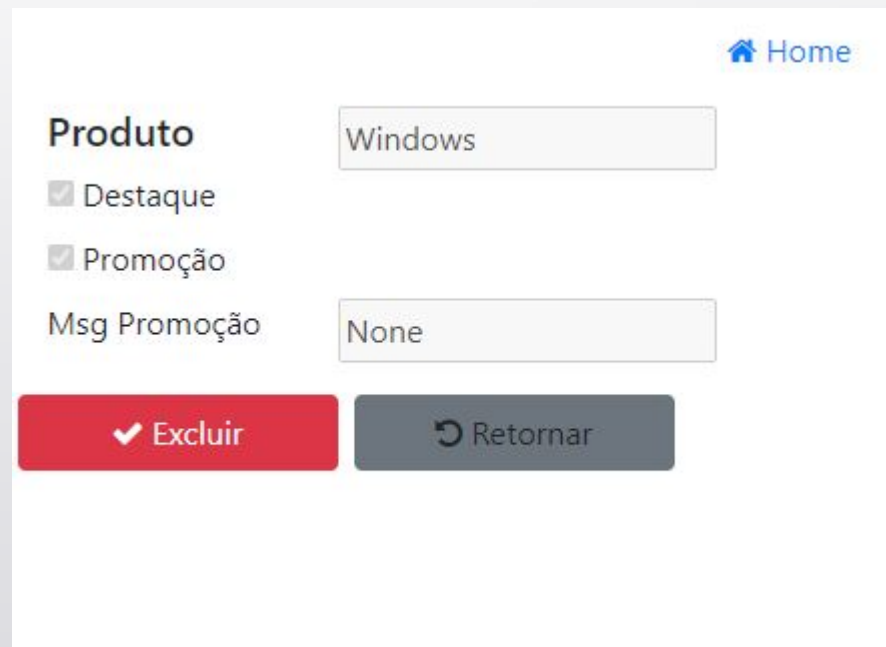
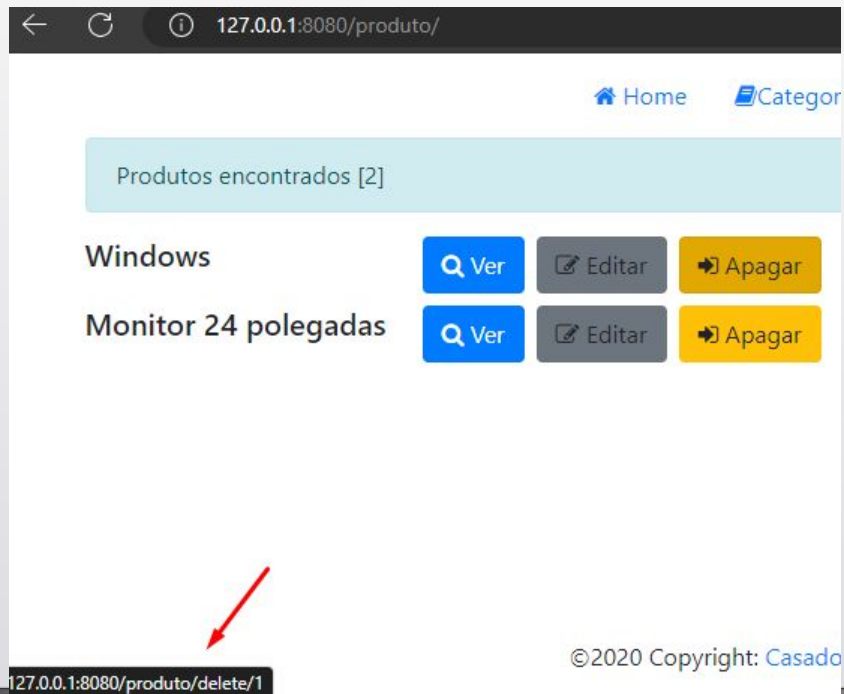
E no template de produto, loja/templates/produto/**produto.html**, altere a url a ser chamada pelo botão de Excluir para produto/delete. Como no exemplo abaixo:

```
{% extends "base.html" %}
[...]
{% for prod in produtos %}
    <div class="row"><h5 class="col-md-3">{{prod.Produto}}</h5>
    <a href={% url 'details_produto' prod.id %} class="btn btn-primary mb-2 mr-2"><i
class="fa fa-search"></i> Ver</a>
    <a href={% url 'edit_produto' prod.id %} class="btn bg-secondary mb-2 mr-2"><i
class="fa fa-edit"></i> Editar</a>
    <a href={% url 'delete_produto' prod.id %} class="btn btn-warning mb-2"><i
class="fa fa-trash"></i> Apagar</a>
    </div>
{% endfor %}
[...]
```

Django

Capítulo 7 - Trabalhando com Templates

Rode a aplicação novamente e confira a aplicação.



Django

Capítulo 7 - Trabalhando com Templates

É necessário agora adicionar a função que vai tratar o postback da interface de excluir pois é ela que efetivamente realiza a exclusão da informação no banco de dados.

Adicione no final do arquivo loja/views/**ProdutoView.py** a funcionalidade de exclusão. Ela será implementada em uma nova função denominada `delete_produto_postback` que será disparada ao clicar o botão de submit da interface de exclusão.

adicione a função que trata o postback da interface de exclusão

```
def delete_produto_postback(request, id=None):
```

```
    # Processa o post back gerado pela action
```

```
    if request.method == 'POST':
```

```
        # Salva dados editados
```

```
        id = request.POST.get("id")
```

```
        produto = request.POST.get("Produto")
```

```
        print("postback-delete")
```

```
        print(id)
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 da função `delete_produto_postback` do arquivo `loja/views/ProdutoView.py`:

```
try:
    Produto.objects.filter(id=id).delete()
    print("Produto %s excluido com sucesso" % produto)
except Exception as e:
    print("Erro salvando edição de produto: %s" % e)
return redirect("/produto")
```

```
loja > views > ProdutoView.py > ...
1  from django.shortcuts import render, redirect
2  from loja.models import Produto
3  from datetime import timedelta, datetime
4  from django.utils import timezone
5  # adicione a função que trata o postback da interface de exclusão
6  def delete_produto_postback(request, id=None):
7      # Processa o post back gerado pela action
8      if request.method == 'POST':
9          # Salva dados editados
10         id = request.POST.get("id")
11         produto = request.POST.get("Produto")
12         print("postback-delete")
13         print(id)
14         try:
15             Produto.objects.filter(id=id).delete()
16             print("Produto %s excluido com sucesso" % produto)
17         except Exception as e:
18             print("Erro salvando edição de produto: %s" % e)
19     return redirect("/produto")
```

Django

Capítulo 7 - Trabalhando com Templates

No arquivo **ProdutoUrls.py** inclua o acesso a função do postback.

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view, edit_produto_view,
edit_produto_postback, details_produto_view, delete_produto_view, delete_produto_postback
urlpatterns = [
    path("", list_produto_view, name='produto'),
    path("<int:id>", list_produto_view, name='produto'),
    path("edit/<int:id>", edit_produto_view, name='edit_produto'),
    path("edit", edit_produto_postback, name='edit_produto_postback'),
    path("details/<int:id>", details_produto_view, name='details_produto'),
    path("delete/<int:id>", delete_produto_view, name='delete_produto'),
    path("delete", delete_produto_postback, name='delete_produto_postback'),
]
```

Django

Capítulo 7 - Trabalhando com Templates

Rode a aplicação e acione o excluir. Após a chamada ao comando de exclusão a lista de produtos irá aparecer sem o produto excluído

The image displays two screenshots of a Django web application interface, illustrating the process of deleting a product.

Left Screenshot (Product Form):

- Produto:** Monitor 24 polegadas
- ☐ Destaque
- ☒ Promoção
- Msg Promoção: Só até sabado
- Buttons:** Excluir (red button with a checkmark icon), Retornar (grey button with a circular arrow icon)

Right Screenshot (Product List):

- Header:** Produtos encontrados [1]
- Product:** Windows
- Buttons:** Ver (blue button with a magnifying glass icon), Editar (grey button with a pencil icon), Apagar (yellow button with a trash can icon)
- URL Bar:** 127.0.0.1:8080/?produto=Monitor
- Search Section:**
 - Header:** Buscar produtos
 - Input:** nome do produto
 - Button:** Buscar (blue button)
 - Footer:** Produtos encontrados [0]

Django

Capítulo 7 - Trabalhando com Templates

Agora realizaremos a operação de Create. Adicione o código abaixo no final do `loja/views/ProdutoView.py` a funcionalidade de carregar a interface de novo produto (Função GET). Ela será implementada em uma nova função denominada `create_produto_view` que será disparada ao clicar o botão de Novo Produto. A implementação inicial dessa função irá apenas chamar o template `produto-create.html` que será criado.

adicione a função que chama a interface de criar produto

no final do arquivo

```
def create_produto_view(request, id=None):
```

```
    return render(request, template_name='produto/produto-create.html', status=200)
```

```
loja > views > ProdutoView.py > ...
1  from django.shortcuts import render, redirect
2  from loja.models import Produto
3  from datetime import timedelta, datetime
4  from django.utils import timezone
5  # adicione a função que chama a interface de criar produto
6  # no final do arquivo
7  def create_produto_view(request, id=None):
8      return render(request, template_name='produto/produto-create.html', status=200)
```


Django

Capítulo 7 - Trabalhando com Templates

Agora inclua no arquivo de urls, loja/urls/**ProdutoUrls.py**, o leiaute da chamada para a interface de criar produto, adicionando no código os trechos destacados, assim:

```
from django.urls import path
from loja.views.ProdutoView import list_produto_view, edit_produto_view,
edit_produto_postback, details_produto_view, delete_produto_view, create_produto_view
urlpatterns = [
    path("", list_produto_view, name= 'produto'),
    path("<int:id>", list_produto_view, name= 'produto'),
    path("edit/<int:id>", edit_produto_view, name= 'edit_produto'),
    path("edit", edit_produto_postback, name= 'edit_produto_postback'),
    path("details/<int:id>", details_produto_view, name= 'details_produto'),
    path("delete/<int:id>", delete_produto_view, name= 'delete_produto'),
    path("create", create_produto_view, name= 'create_produto'),
]
```

Django

Capítulo 7 - Trabalhando com Templates

Para tal será incluído um botão Novo Produto na interface de produto, loja/templates/produto/produto.html, que dará acesso a interface de criar produto. Altere o arquivo produto.html para inserir o botão. Esse botão irá chamar a url create_produto e será utilizado o icone fa-plus.

```
{% extends "base.html" %}
[...]
<div id="content">
    <div class="container">
        <div class="alert alert-info">
            <a href={% url 'create_produto' %} class="btn btn-primary mr-2"> <i class="fa
fa-plus"></i> Novo Produto</a>
            Produtos encontrados [{{ produtos | length }}]
        </div>
        {% for prod in produtos %}
        [...]
```

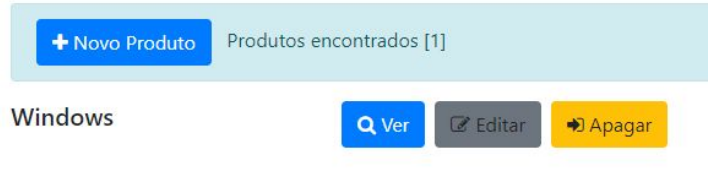
Django

Capítulo 7 - Trabalhando com Templates

Trecho do código com a modificação:

```
loja > templates > produto > <> produto.html > ...
```

```
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Início{% endblock %}
4  {% block styles %}
5  <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8  <div id="content">
9      <div class="container">
10         <div class="alert alert-info">
11             <a href="{% url 'create_produto' %}" class="btn btn-primary mr-2"> <i class="fa fa-plus"></i> Novo Produto</a>
12             Produtos encontrados [{% produtos | length %}]
13         </div>
14         {% for prod in produtos %}
15         <div class="row">
16             <div class="col-md-3">{{prod.Produto}}</div>
17             <a href="{% url 'details_produto' prod.id %}" class="btn btn-primary mb-2 mr-2"><i class="fa fa-search"></i> Ver</a>
18             <a href="{% url 'edit_produto' prod.id %}" class="btn btn-secondary mb-2 mr-2"><i class="fa fa-edit"></i> Editar</a>
19             <a href="{% url 'delete_produto' prod.id %}" class="btn btn-warning mb-2"><i class="fa fa-sign-in"></i> Apagar</a>
20         </div>
21         {% endfor %}
22     </div>
23 </div>
24 {% endblock %}
25 {% block scripts %}
26 <script src="{% static 'js/home.js' %}"></script>
27 {% endblock %}
```



Django

Capítulo 7 - Trabalhando com Templates

Agora é preciso criar na pasta template de produto o arquivo html que permite informar os dados do produto a ser incluído. Nessa interface haverá o botão de inserir para realizar a operação. Por simplificação iremos copiar a interface utilizada na interface de edição e ajustar a funcionalidade do botão. Faça uma cópia do arquivo produto-edit.html com nome produto-create.html e faça os seguintes ajustes:

- (1) inclua no form a opção `enctype="multipart/form-data"` para informar que vai ser passados arquivos,
- (2) ajuste a action do form para `"/produto/create"`,
- (3) Inclua campos para o preço e a imagem do produto,
- (4) retire o campo de id do produto, os comandos if-else dos checkbox e os values dos demais e
- (5) substitua o texto do botão de Salvar para Inserir. As alterações estão destacadas no código a seguir.

Django

Capítulo 7 - Trabalhando com Templates

Parte 1 do código do loja/templates/produto/**produto-create.html**:

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
    <link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
<div id="content">
    <div class="container">
        <form method="POST" enctype="multipart/form-data" action= "/produto/create" >
            {% csrf_token %}
            <div class="row">
                <!--<input type="hidden" value="{{produto.id}}" name="id">-->
                <h5 for="Produto" class="col-md-2">Produto</h5>
                <input name="Produto">
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do código do loja/templates/produto/**produto-create.html**:

```
</div>
<div class="row">
  <div class="col-md-2">
    <input type="checkbox" name="destaque" >
    <label for="destaque">Destaque</label>
  </div>
</div>
<div class="row">
  <div class="col-md-2">
    <input type="checkbox" name="promocao" >
    <label for="promocao">Promoção</label>
  </div>
</div>
<div class="row">
  <label for="msgPromocao" class="col-md-2">Msg Promoção</label>
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 do código do loja/templates/produto/**produto-create.html**:

```
        <input name="msgPromocao">
    </div>
    <div class="row mt-2">
        <label for="preco" class="col-md-2">Preço</label>
        <input name="preco">
    </div>
    <div class="row mt-2">
        <label for="image" class="col-md-2">Imagem</label>
        <input type="file" name="image">
    </div>
    <div class="row">
        <button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i
class="fa fa-check"></i> Inserir</button>
        <a href={% url 'produto' %} class="btn bg-secondary col-md-2 mt-3"><i
class="fa fa-rotate-left"></i> Retornar</a>
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 4 do código do loja/templates/produto/**produto-create.html**:

```
        </div>
    </form>
</div>
</div>
{% endblock %}
{% block scripts %}
    <script src="{% static 'js/home.js' %}"></script>
{% endblock %}
```


Django

Capítulo 7 - Trabalhando com Templates

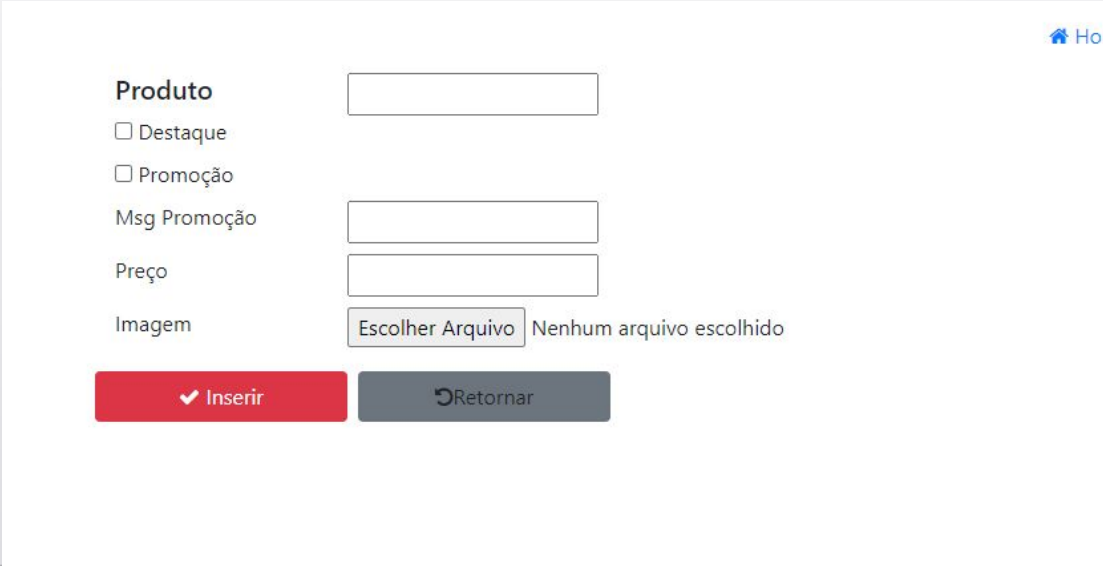
Código do loja/templates/produto/**produto-create.html**:

```
loja > templates > produto > <- produto-create.html > ...
1 {% extends "base.html" %}
2 {% load static %}
3 {% block title %}Inicio{% endblock %}
4 {% block styles %}
5 <link rel="stylesheet" href="{% static 'css/home.css' %}">
6 {% endblock %}
7 {% block content %}
8 <div id="content">
9     <div class="container">
10         <form method="POST" enctype="multipart/form-data" action="/produto/create">
11             {% csrf_token %}
12             <div class="row">
13                 <!--input type="hidden" value="{{[produto.id]}}" name="id"-->
14                 <h5 form="Produto" class="col-md-2">Produto</h5>
15                 <input name="Produto">
16             </div>
17             <div class="row">
18                 <div class="col-md-2">
19                     <input type="checkbox" name="destaque">
20                     <label form="destaque">Destaque</label>
21                 </div>
22             </div>
23             <div class="row">
24                 <div class="col-md-2">
25                     <input type="checkbox" name="promocao">
26                     <label form="promocao">Promoção</label>
27                 </div>
28             </div>
29             <div class="row">
30                 <label form="msgPromocao" class="col-md-2">Msg Promoção</label>
31                 <input name="msgPromocao">
32             </div>
33             <div class="row mt-2">
34                 <label form="preco" class="col-md-2">Preço</label>
35                 <input name="preco">
36             </div>
37             <div class="row mt-2">
38                 <label form="image" class="col-md-2">Imagem</label>
39                 <input type="file" name="image">
40             </div>
41             <div class="row">
42                 <button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i class="fa fa-check"></i>
43                 <button
44                     <a href="{% url 'produto' %}" class="btn bg-secondary col-md-2 mt-3"><i class="fa fa-rotate-left"></i>Retornar</a>
45             </div>
46         </form>
47     </div>
48 </div>
49 {% endblock %}
50 {% block scripts %}
51 <script src="{% static 'js/home.js' %}"></script>
52 {% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Execute a aplicação e observe o botão de Novo produto e visualize a interface de criar produto. A interface ainda não está salvando pois não foi criado ainda o tratamento para o evento POST da interface.



The screenshot displays a web form for creating a new product. The form is titled "Produto" and includes several input fields and checkboxes. The "Produto" field is a single-line text input. Below it are two checkboxes: "Destaque" and "Promoção". The "Msg Promoção" field is a single-line text input. The "Preço" field is a single-line text input. The "Imagem" field is a file upload area with a button labeled "Escolher Arquivo" and the text "Nenhum arquivo escolhido". At the bottom of the form are two buttons: a red button labeled "✓ Inserir" and a grey button labeled "↶ Retornar". In the top right corner, there is a small blue icon and the text "Hoi".

Produto

☐ Destaque

☐ Promoção

Msg Promoção

Preço

Imagem

Escolher Arquivo Nenhum arquivo escolhido

✓ Inserir

↶ Retornar

Django

Capítulo 7 - Trabalhando com Templates

É necessário agora adicionar o tratamento do postback da interface de criar para que as informações digitadas sejam efetivamente inseridas no banco de dados.

Observe que na implementação do Edit e do Delete a função GET tinha parâmetros e a função POST não tinha. No caso do CREATE nenhum dos dois tipos utiliza parâmetros na URL portanto vão compartilhar a mesma função. Dessa forma será necessário alterar a função já existente `create_produto_view` para que ela faça a funcionalidade de POST. Altere no arquivo `loja/views/ProdutoView.py` a função referida com as alterações destacadas a seguir.

```
from django.shortcuts import render, redirect
from loja.models import Produto
from datetime import timedelta, datetime
from django.utils import timezone
# inclua as bibliotecas FileSystemStorage
from django.core.files.storage import FileSystemStorage
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 1 da modificação da função do loja/views/**ProdutoView.py**:

```
[...]  
def create_produto_view(request, id=None):  
    # Processa o post back gerado pela action  
    if request.method == 'POST':  
        produto = request.POST.get("Produto")  
        destaque = request.POST.get("destaque")  
        promocao = request.POST.get("promocao")  
        msgPromocao = request.POST.get("msgPromocao")  
        preco = request.POST.get("preco")  
        image = request.POST.get("image")  
        print("postback-create")  
        print(produto)  
        print(destaque)  
        print(promocao)
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 da modificação da função do loja/views/**ProdutoView.py**:

```
print(msgPromocao)
print(preco)
print(image)
try:
    obj_produto = Produto()
    obj_produto.Produto = produto
    obj_produto.destaque = (destaque is not None)
    obj_produto.promocao = (promocao is not None)
    if msgPromocao is not None:
        obj_produto.msgPromocao = msgPromocao
    obj_produto.preco = 0
    if (preco is not None) and (preco != ""):
        obj_produto.preco = preco
    obj_produto.criado_em = timezone.now()
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 da modificação da função do loja/views/**ProdutoView.py**:

```
obj_produto.alterado_em = obj_produto.criado_em
# Se for anexado arquivo, salva na pasta e guarda nome no objeto
if request.FILES is not None:
    num_files = len(request.FILES.getlist('image'))
    if num_files > 0:
        imagefile = request.FILES['image']
        print(imagefile)
        fs = FileSystemStorage()
        filename = fs.save(imagefile.name, imagefile)
        if (filename is not None) and (filename != ""):
            obj_produto.image = filename
obj_produto.save()
print("Produto %s salvo com sucesso" % produto)
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 4 da modificação da função do loja/views/**ProdutoView.py**:

```
except Exception as e:
    print("Erro inserindo produto: %s" % e)
    return redirect("/produto")
return render(request, template_name='produto/produto-create.html', status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Código modificado do loja/views/**ProdutoView.py**:

```
loja > views > ProdutoView.py > ...
1 from django.shortcuts import render, redirect
2 from loja.models import Produto
3 from datetime import timedelta, datetime
4 from django.utils import timezone
5 # inclui as bibliotecas FileSystemStorage
6 from django.core.files.storage import FileSystemStorage
7 def create_produto_view(request, isAjax=None):
8     #Processa o post back gerado pela action
9     if request.method == 'POST':
10         produto = request.POST.get("Produto")
11         destaque = request.POST.get("destaque")
12         promocao = request.POST.get("promocao")
13         msgPromocao = request.POST.get("msgPromocao")
14         preco = request.POST.get("preco")
15         image = request.POST.get("image")
16         print("postback-create")
17         print(produto)
18         print(destaque)
19         print(promocao)
20         print(msgPromocao)
21         print(preco)
22         print(image)
23         try:
24             obj_produto = Produto()
25             obj_produto.Produto = produto
26             obj_produto.destaque = (destaque is not None)
27             obj_produto.promocao = (promocao is not None)
28             if msgPromocao is not None:
29                 obj_produto.msgPromocao = msgPromocao
30             obj_produto.preco = 0
31             if (preco is not None) and (preco != ""):
32                 obj_produto.preco = preco
33             obj_produto.criado_em = timezone.now()
34             obj_produto.alterado_em = obj_produto.criado_em
35             # Se for anexado arquivo, salva na pasta e guarda nome no objeto
36             if request.FILES is not None:
37                 num_files = len(request.FILES.getlist('image'))
38                 if num_files > 0:
39                     imagefile = request.FILES['image']
40                     print(imagefile)
41                     fs = FileSystemStorage()
42                     filename = fs.save(imagefile.name, imagefile)
43                     if (filename is not None) and (filename != ""):
44                         obj_produto.image = filename
45             obj_produto.save()
46             print("Produto %s salvo com sucesso" % produto)
47         except Exception as e:
48             print("Erro inserindo produto: %s" % e)
49         return redirect("/produto")
50     return render(request, template_name='produto/produto-create.html', status=200)
```


Django

Capítulo 7 - Trabalhando com Templates

Roda a aplicação e crie um produto

Produto

☐ Destaque

☒ Promoção

Msg Promoção

Preço

Imagem peripherals_...gallery_1.jpg

Hc

+ Novo Produto

Produtos encontrados [2]

Windows

Q Ver

✎ Editar

🗑 Apagar

Monitor 12 polegadas

Q Ver

✎ Editar

🗑 Apagar

Produto

☐ Destaque

☒ Promoção

Msg Promoção



↶ Retornar

Django

Capítulo 7 - Trabalhando com Templates

Até o momento só salvamos produtos sem associação com a categoria ou fabricante. Agora iremos adicionar a relação das entidades na edição do produto. Primeiro, inclua acesso aos modelos de fabricante e categoria no arquivo loja/views/**ProdutoView.py**, assim:

```
from django.shortcuts import render, redirect
#Inclua as classes modelos de fabricante e categoria
from loja.models import Produto, Fabricante, Categoria
from datetime import timedelta, datetime
from django.utils import timezone
from django.core.files.storage import FileSystemStorage
[...]
```

```
from django.shortcuts import render, redirect
#Inclua as classes modelos de fabricante e categoria
from loja.models import Produto, Fabricante, Categoria
from datetime import timedelta, datetime
from django.utils import timezone
from django.core.files.storage import FileSystemStorage
```

Django

Capítulo 7 - Trabalhando com Templates

Agora vamos ajustar a função da visão do edit, *edit_produto_view* no `loja/views/ProdutoView.py`, para exibir as listas em forma de select para que possamos escolher qual o fabricante e categoria do produto

```
def edit_produto_view(request, id=None):
    produtos = Produto.objects.all()
    if id is not None:
        produtos = produtos.filter(id=id)
    produto = produtos.first()
    print(produto)
    #adicione a lista de fabricantes e categorias no context
    Fabricantes = Fabricante.objects.all()
    Categorias = Categoria.objects.all()
    context = {'produto': produto, 'fabricantes' : Fabricantes, 'categorias' : Categorias}
    return render(request, template_name='produto/produto-edit.html', context=context,
status=200)
```

Django

Capítulo 7 - Trabalhando com Templates

Já podemos modificar o template para vermos as listas adicionadas em um select. Para isso vá para loja/templates/produto/**produto-edit.html** e adicione os seguintes divs destacados:

```
[...]
<div class="row">
    <label for="msgPromocao" class="col-md-2">Msg Promoção</label>
    <input value="{{produto.msgPromocao}}" name="msgPromocao">
</div>
<div class="row">
    <label for="CategoriaFk" class="col-md-2" >Categorias</label>
    <select name="CategoriaFk">
        <option value="-1">-----</option>
        {% for cate in categorias %}
            {% if produto.categoria_id == cate.id %}
                <option value="{{cate.id}}" selected>{{cate.Categoria}}</option>
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 do loja/templates/produto/**produto-edit.html**:

```
        {% else %}
        <option value="{{cate.id}}">{{cate.Categoria}}</option>
        {% endif %}
    {% endfor %}
</select>
</div>
<div class="row">
    <label for="FabricanteFk" class="col-md-2" >Fabricantes</label>
    <select name="FabricanteFk">
        <option value="-1">-----</option>
        {% for fabri in fabricantes %}
            {% if produto.fabricante_id == fabri.id %}
                <option value="{{fabri.id}}"
selected>{{fabri.Fabricante}}</option>
            {% else %}
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 3 do loja/templates/produto/**produto-edit.html**:

```
                <option value="{{fabri.id}}">{{fabri.Fabricante}}</option>
                {% endif %}
            {% endfor %}
        </select>
    </div>
    <div class="row">
        <button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i
class="fa fa-check"></i>Salvar</button>
        <a href={% url 'produto' %} class="btn bg-secondary col-md-2 mt-3"><i
class="fa fa-rotate-left"></i>Retornar</a>
    </div>
    [...]
```

Django

Capítulo 7 - Trabalhando com Templates

Código do loja/templates/produto/**produto-edit.html**:

```
37 <div class="row">
38   <label for="msgPromocao" class="col-md-2">Msg Promoção</label>
39   <input value="{{produto.msgPromocao}}" name="msgPromocao">
40 </div>
41 <div class="row">
42   <label for="CategoriaFk" class="col-md-2">Categorias</label>
43   <select name="CategoriaFk">
44     <option value="-1">-----</option>
45     {% for cate in categorias %}
46       {% if produto.categoria_id == cate.id %}
47       <option value="{{cate.id}}" selected>{{cate.Categoria}}</option>
48       {% else %}
49       <option value="{{cate.id}}">{{cate.Categoria}}</option>
50       {% endif %}
51     {% endfor %}
52   </select>
53 </div>
54 <div class="row">
55   <label for="FabricanteFk" class="col-md-2">Fabricantes</label>
56   <select name="FabricanteFk">
57     <option value="-1">-----</option>
58     {% for fabri in fabricantes %}
59       {% if produto.fabricante_id == fabri.id %}
60       <option value="{{fabri.id}}" selected>{{fabri.Fabricante}}</option>
61       {% else %}
62       <option value="{{fabri.id}}">{{fabri.Fabricante}}</option>
63       {% endif %}
64     {% endfor %}
65   </select>
66 </div>
67 <div class="row">
68   <button type="submit" class="btn btn-danger col-md-2 mt-3 mr-2"><i class="fa fa-check"></i> Salvar</button>
69   <a href="{% url 'produto' %}" class="btn bg-secondary col-md-2 mt-3"><i class="fa fa-rotate-left"></i> Retornar</a>
70 </div>
71 </form>
72 </div>
73 </div>
74 {% endblock %}
75 {% block scripts %}
76 <script src="{% static 'js/home.js' %}"></script>
77 {% endblock %}
```

Django

Capítulo 7 - Trabalhando com Templates

Rode e confira os selects adicionados.

Produto
☐ Destaque
☒ Promoção
Msg Promoção
Categorias
Fabricantes

----- ▼

----- ▼

Dell
Microsoft

Produto
☒ Destaque
☒ Promoção
Msg Promoção
Categorias
Fabricantes

Software ▼

Microsoft ▼

Django

Capítulo 7 - Trabalhando com Templates

Agora iremos voltar no `loja/views/ProdutoView.py` e modifique o `edit_produto_postback` para salvar o fabricante e a categoria do produto. Para isso adicione os techos de códigos destacados a seguir:

```
def edit_produto_postback(request, id=None):  
    if request.method == 'POST':  
        id = request.POST.get("id")  
        produto = request.POST.get("Produto")  
        destaque = request.POST.get("destaque")  
        promocao = request.POST.get("promocao")  
        msgPromocao = request.POST.get("msgPromocao")  
        #adicione a requisição do valor do campo post  
        categoria = request.POST.get("CategoriaFk")  
        fabricante = request.POST.get("FabricanteFk")  
        print("postback")  
        print(id)
```

Django

Capítulo 7 - Trabalhando com Templates

Parte 2 da função `edit_produto_postback` do `loja/views/ProdutoView.py`:

```
print(produto)
print(destaque)
print(promocao)
print(msgPromocao)
try:
    obj_produto = Produto.objects.filter(id=id).first()
    obj_produto.Produto = produto
    obj_produto.destaque = (destaque is not None)
    obj_produto.promocao = (promocao is not None)
    #salve os objeto fabricante e categoria filtrados com base no id recebido na
    #variavel categoria e fabricante
    obj_produto.fabricante = Fabricante.objects.filter(id=fabricante).first()
    obj_produto.categoria = Categoria.objects.filter(id=categoria).first()
    if msgPromocao is not None:
        [...]
```

Django

Capítulo 7 - Trabalhando com Templates

Código do `edit_produto_postback` do `loja/views/ProdutoView.py`:

```
def edit_produto_postback(request, id=None):
    if request.method == 'POST':
        id = request.POST.get("id")
        produto = request.POST.get("Produto")
        destaque = request.POST.get("destaque")
        promocao = request.POST.get("promocao")
        msgPromocao = request.POST.get("msgPromocao")
        #adicione a requisicao do valor do campo post
        categoria = request.POST.get("CategoriaFk")
        fabricante = request.POST.get("FabricanteFk")
        print("postback")
        print(id)
        print(produto)
        print(destaque)
        print(promocao)
        print(msgPromocao)
    try:
        obj_produto = Produto.objects.filter(id=id).first()
        obj_produto.Produto = produto
        obj_produto.destaque = (destaque is not None)
        obj_produto.promocao = (promocao is not None)
        #salve os objeto fabricante e categoria filtrados com base no id recebido na variavel categoria e fabricante
        obj_produto.fabricante = Fabricante.objects.filter(id=fabricante).first()
        obj_produto.categoria = Categoria.objects.filter(id=categoria).first()

        if msgPromocao is not None:
            obj_produto.msgPromocao = msgPromocao
            obj_produto.save()
            print("Produto %s salvo com sucesso" % produto)
    except Exception as e:
        print("Erro salvando edição de produto: %s" % e)
    return redirect("/produto")
```

Django

Capítulo 7 - Trabalhando com Templates

Agora já podemos editar o fabricante e a categoria do produto. Rode e verifique a funcionalidade.

The image displays two screenshots of a Django web application interface for managing products. The interface includes a sidebar with navigation links: 'Produto', 'Destaque', 'Promoção', 'Msg Promoção', 'Categorias', and 'Fabricantes'. The main content area shows a product form for 'Monitor'.

Left Screenshot: This view shows the product form with a dropdown menu for 'Categorias' open, displaying options like 'Hardware', 'Dell', and 'Microsoft'. Red arrows point to the 'Monitor' text field, the 'Hardware' option in the category dropdown, and the 'Microsoft' option in the manufacturer dropdown. A '+ Novo Produto' button is visible in the top right of the main area.

Right Screenshot: This view shows the same product form after the changes have been saved. The 'Categorias' dropdown now shows 'Hardware' and the 'Fabricantes' dropdown shows 'Microsoft'. Red arrows point to the 'Monitor' text field and the 'Hardware' option in the category dropdown. The 'Destaque' checkbox is unchecked, and the 'Promoção' checkbox is checked.

Product Form Fields:

- Produto:** Monitor
- ☐ Destaque
- ☒ Promoção
- Msg Promoção: Ate hoje!
- Categorias: Hardware
- Fabricantes: Microsoft
-
-